GUÍA COMPLETA DEL EDA

¿Cuánto vale tu trabajo?

De cero absoluto a experto: todo lo que necesitas saber para entender, explicar y replicar este proyecto

38 países **8 datasets** **1.016 filas** **5 hipótesis**

Andoni Olaso | Data Science Bootcamp | Abril 2026

Índice

01 Conceptos fundamentales (qué es un EDA, un dataset, una hipótesis...)

02 El proyecto: de dónde nace y qué quiero averiguar

03 Los datos: qué son, de dónde salen, qué significan

04 Notebook 1: cómo limpié y preparé los datos (paso a paso)

05 Notebook 2: análisis de cada hipótesis (con cada gráfico explicado)

06 La sección del País Vasco

07 La web: qué muestra cada sección

08 Chuleta rápida: comandos y conceptos clave

01. Conceptos fundamentales

Antes de entrar en el proyecto, necesitas entender qué significan las palabras que vas a leer. Esta sección explica cada concepto desde cero, como si nunca hubieras oído hablar de data science.

¿Qué es Data Science?

Data Science (ciencia de datos) es el proceso de **extraer conocimiento útil a partir de datos**. Es como ser un detective, pero en vez de interrogar testigos, interrogas números y tablas. Tu objetivo es encontrar patrones, relaciones y verdades que no son obvias a simple vista.

¿Qué es un dataset?

Dataset = tabla de datos

Un dataset es simplemente una **tabla**, como las de Excel. Tiene **filas** (cada fila es una "observación" o un caso) y **columnas** (cada columna es una "variable" o un tipo de información).

Ejemplo: imagina una tabla de alumnos de una clase:

Nombre	Edad	Nota	Ciudad
Ana	22	8.5	Madrid
Jon	25	7.2	Bilbao
Marta	21	9.0	Barcelona

Esto es un dataset. Tiene 3 filas (3 alumnos) y 4 columnas (nombre, edad, nota, ciudad). En este proyecto, mi dataset final tiene **1.016 filas** (combinaciones de país y año) y **12 columnas** (salario, vivienda, Gini, etc.).

¿Qué es un EDA?

EDA = Análisis Exploratorio de Datos

EDA son las siglas de *Exploratory Data Analysis*. Es el **primer paso** que se hace cuando tienes datos: antes de crear modelos complejos o predicciones, necesitas **mirar los datos, entenderlos, limpiarlos y buscar patrones**.

Es como cuando un médico te hace un chequeo general antes de operarte: primero mira, escucha, toca, y recién entonces decide qué hacer.

Analogía: *Imagina que compras una casa vieja. Antes de reformarla (el modelo predictivo), necesitas inspeccionar cada habitación, comprobar si hay humedades, ver el estado de las tuberías, medir las paredes. Eso es el EDA: la inspección completa antes de tomar decisiones.*

Un EDA tiene estas fases:

1. **Definir preguntas:** ¿Qué quiero averiguar? Planteo hipótesis ANTES de tocar los datos.
2. **Obtener datos:** Busco fuentes fiables (en mi caso, Eurostat).
3. **Explorar:** Miro las primeras filas, cuántos nulos hay, qué tipos de datos tengo

3 **Explorar:** Miro las primeras filas, cuántos nulos hay, qué tipos de datos tengo.

4 **Limpiar:** Arreglo errores, formatos raros, valores faltantes. Esta es la parte más larga (el 80% del trabajo).

5 **Analizar:** Hago gráficos y tests estadísticos para responder mis preguntas.

6 **Comunicar:** Presento los resultados de forma que se entiendan.

¿Qué es una hipótesis?

> **Hipótesis = una afirmación que puedo comprobar con datos**
>
> Una hipótesis es una **frase que dice algo concreto sobre la realidad** y que puedo verificar si es verdad o mentira mirando los datos. No es una opinión ni una intuición: es algo que se puede medir.
>
> **Ejemplo de hipótesis buena:** "Los países con salario mínimo más alto tienen menos pobreza laboral."
>
> **Ejemplo de hipótesis mala:** "El salario mínimo debería ser más alto." (Esto es una opinión, no se puede medir con datos.)
>
> En un EDA, las hipótesis se plantean ANTES de analizar los datos. ¿Por qué? Para evitar el sesgo de confirmación: si primero miras los datos y luego "descubres" un patrón, es fácil engañarte. Si planteas la hipótesis antes, los datos te corrigen si estás equivocado.

¿Qué es una correlación?

> **Correlación = ¿dos cosas se mueven juntas?**
>
> La correlación mide si **cuando una variable sube, la otra también sube (o baja)**. Nada más. NO dice que una CAUSE la otra.

> **Analogía:** *Hay correlación entre las ventas de helados y los ahogamientos en piscina. Cuando suben los helados, suben los ahogamientos. ¿Los helados causan ahogamientos? No. Ambos suben porque hace calor. El calor es la causa real. La correlación solo dice "se mueven juntos", no dice "uno causa el otro".*

¿Qué es una relación lineal?

Una relación es **lineal** cuando, si la dibujas en un gráfico, los puntos se alinean más o menos en una **línea recta**. Si forman una curva, una U, o un patrón más complejo, la relación es **no lineal**.

> **Analogía:** *Si cada vez que estudias 1 hora más, tu nota sube 0.5 puntos, eso es lineal: más estudio = más nota, siempre en la misma proporción. Pero si estudiar 1-3 horas te sube mucho la nota y estudiar 8-10 horas ya no te sube nada (porque estás agotado), eso es no lineal: la relación cambia de forma.*

¿Por qué importa esto? Porque el test de Pearson (que uso en este proyecto) **solo detecta relaciones lineales**. Si la relación entre dos variables es curva, Pearson puede decir "no hay relación" cuando en realidad sí la hay, solo que no es una línea recta. Esto es exactamente lo que pasa en la Hipótesis 3.

¿Qué es el test de Pearson?

> **Pearson = la herramienta para medir correlaciones lineales**

El test de Pearson te da **dos números**:

1. r (coeficiente de correlación): un número entre -1 y +1.

- **r = +1:** correlación perfecta positiva. Cuando A sube, B sube exactamente igual. Como la temperatura y las ventas de helados.
- **r = 0:** no hay correlación. A y B no tienen nada que ver. Como tu número de zapato y tu nota en mates.
- **r = -1:** correlación perfecta negativa. Cuando A sube, B baja exactamente igual. Como la altura a la que estás y la temperatura (más alto = más frío).

2. p-value (valor p): la probabilidad de que el resultado sea casualidad.

- **p < 0.05:** resultado **significativo**. Menos del 5% de probabilidad de que sea azar. Puedes fiarte.
- **p > 0.05:** resultado **no significativo**. Podría ser casualidad. No puedes fiarte.

Analogía del p-value: *Imagina que tiras una moneda al aire 100 veces. Si sale cara 52 veces, no es raro (p alto, podría ser casualidad). Si sale cara 95 veces, algo pasa con esa moneda (p bajo, no es casualidad). El p-value te dice: ¿este resultado es tan extremo que no puede ser suerte?*

Interpretación rápida de r:

Valor de |r|	¿Qué significa?	Ejemplo del proyecto
0.8 - 1.0	Relación muy fuerte	H1: EUR vs PPS (r = 0.945)
0.5 - 0.8	Relación fuerte	(ninguna en este EDA)
0.3 - 0.5	Relación moderada	(ninguna en este EDA)
0.1 - 0.3	Relación débil	H2: r = -0.299, H4: r = -0.245
0 - 0.1	Relación muy débil o nula	H3: r = 0.044, H5: r = -0.060

Cuidado: un p-value alto (no significativo) NO significa que no exista relación. Puede significar tres cosas: (1) la muestra es pequeña, (2) la relación no es lineal (y Pearson solo mide lineales), o (3) realmente no hay relación. En H3, la relación existe pero no es lineal: la pregunta no es "¿más salario = más vivienda?" sino "¿cuál crece más rápido?" Eso no se mide con Pearson.

¿Qué es Python? ¿Y Pandas?

Python = el lenguaje, Pandas = la herramienta para tablas

Python es un lenguaje de programación. Es como dar instrucciones a un ordenador en un idioma que entiende. En vez de decirle "abre el Excel y filtra por España", le escribes una línea de código que hace lo mismo pero más rápido y sin errores.

Pandas es una "librería" de Python (un paquete de herramientas extra) especializada en trabajar con tablas de datos. Es como Excel pero con superpoderes: puede manejar millones de filas, hacer operaciones complejas en una línea, y no se cuelga.

Jupyter Notebook es el programa donde escribo el código. Es como un documento de Word donde puedes mezclar texto explicativo con código que se ejecuta. Los resultados (tablas, gráficos) aparecen justo debajo de cada bloque de código.

02. El proyecto: de dónde nace

La pregunta central

Todo nace de una pregunta simple: **¿cuánto vale realmente un salario mínimo?**

No cuántos euros dice la nómina, sino qué puedes comprar con ese dinero. Porque 1.000€ en Madrid no son lo mismo que 1.000€ en Dublín (donde la vida es carísima) o en Bucarest (donde la vida es barata).

Esta pregunta se conecta con un debate político real: la propuesta de subir el salario mínimo a 1.500€ en el País Vasco y en España. ¿Es mucho? ¿Es poco? Solo lo puedes saber si lo comparas con lo que pasa en el resto de Europa.

Las 5 hipótesis

Antes de tocar los datos, planteé 5 afirmaciones concretas que quería comprobar. Cada una es una pregunta que se puede responder con números:

#	Hipótesis	Lo que pregunto en cristiano	Qué datos necesito
H1	El salario nominal engaña	¿1.000€ compran lo mismo en todos los países?	Salario en EUR y en PPS
H2	SM como herramienta feminista	¿Subir el SM reduce la brecha hombre-mujer?	Salario PPS y brecha de género
H3	El SM no da para vivir	¿La vivienda sube más rápido que el sueldo?	Salario PPS e índice de vivienda
H4	Trabajar no te saca de pobre	¿Tener trabajo garantiza no ser pobre?	Salario PPS y pobreza laboral
H5	La Gen Z vive peor	¿La inflación se come lo que ganas?	Salario EUR e inflación HICP

¿Por qué importa plantear las hipótesis ANTES?

> **Para evitar el sesgo de confirmación.** Si primero miras los datos y luego "descubres" un patrón, tu cerebro te puede engañar: ves lo que quieres ver. Si planteas la hipótesis antes, los datos te corrigen si estás equivocado. En este proyecto, H5 me corrigió: esperaba que la Gen Z viviera peor, pero los datos dicen que el salario real ha SUBIDO en casi toda Europa. Si no hubiera planteado la hipótesis antes, probablemente habría buscado (y encontrado) algún dato que confirmara mi prejuicio.

03. Los datos: qué son y de dónde salen

¿Qué es Eurostat?

Eurostat es la oficina estadística de la Unión Europea. Es la versión europea del INE (Instituto Nacional de Estadística de España). Publica datos gratuitos sobre todos los países de la UE y algunos vecinos. Son los datos oficiales que usan los gobiernos para tomar decisiones.

Los datos se pueden obtener de dos formas:

1. **Descarga manual:** vas a la web de Eurostat, buscas el dataset, aplicas filtros, y descargas un archivo .tsv (parecido a un .csv, pero separado por tabulaciones en vez de comas).
2. **API pública:** le pides los datos al servidor de Eurostat directamente desde Python. Útil cuando necesitas datos históricos que no están en la descarga manual.

¿Qué es PPS? (Purchasing Power Standard)

> **PPS = moneda ficticia que mide lo que puedes COMPRAR**
>
> PPS es una unidad inventada por Eurostat para comparar el poder adquisitivo entre países. **NO se mide en euros.** Es su propia unidad.

> **Analogía:** *Imagina que inventas una moneda llamada "BigMac". Un BigMac vale 1 BigMac en todos los países. Si en España un BigMac cuesta 5€ y en Suiza cuesta 10€, entonces 1.000€ en España = 200 BigMacs, pero 1.000€ en Suiza = solo 100 BigMacs. El PPS hace exactamente eso: traduce los euros de cada país a "cuántas cosas puedes comprar", no "cuántos billetes tienes".*

Ejemplo concreto del proyecto:

- España: salario mínimo = 1.323€ = 1.456 PPS. Cada euro rinde un 10% más de lo esperado (la vida es algo más barata que la media UE).
- Irlanda: salario mínimo = 2.146€ = 1.554 PPS. A pesar de cobrar 823€ MÁS que España en euros, en poder de compra la diferencia se reduce a solo 98 PPS. Irlanda es carísima.
- Rumanía: salario mínimo = 703€ = 1.105 PPS. Cobra 620€ MENOS que España en euros, pero en PPS la diferencia es de solo 351. La vida es muy barata allí.

Los 8 datasets: uno por uno

Usé 8 datasets diferentes. Cada uno aporta una pieza del puzzle:

Dataset 1: Salario mínimo (earn_mw_cur)

Qué mide: El salario mínimo mensual bruto de cada país, en euros (EUR) y en poder adquisitivo (PPS).

De dónde lo saqué: API de Eurostat (el archivo descargable solo tenía ~10 periodos; necesitaba desde 1999).

Columnas que genera: `salario_min_eur` y `salario_min_pps`.

Detalle técnico: Los datos vienen en semestres (S1 = enero, S2 = julio). Calculé la media de ambos semestres para obtener un valor anual.

Dataset 2: PIB per cápita (tec00114)

Qué mide: La riqueza media por habitante de cada país, en PPS.

Para qué lo uso: Como contexto. No lo analizo directamente en las hipótesis, pero me ayuda a entender el nivel económico de cada país.

Columna: `pib_pps`.

Dataset 3: Precios de vivienda (prc_hpi_a)

Qué mide: Un índice que dice cuánto han subido los precios de la vivienda desde un año base (2010 = 100). Si el índice es 150, los precios han subido un 50% desde 2010.

Importante: NO me dice el precio de un piso. Me dice la TENDENCIA: ¿suben o bajan?

Columna: `indice_vivienda`.

Cuidado: Este dataset tenía 9 combinaciones por país/año (3 tipos de vivienda × 3 tipos de unidad). Tuve que filtrar a una sola combinación (TOTAL + I10_A_AVG) para evitar el producto cartesiano (ver Parte 4).

Dataset 4: Coeficiente de Gini (ilc_di12)

Qué mide: La desigualdad de ingresos en un país. Va de 0 (todo el mundo cobra igual) a 100 (una persona tiene todo). Europa va de ~23 (nórdicos, muy iguales) a ~40 (Turquía, Bulgaria, muy desiguales).

Columna: `gini`.

Dataset 5: Brecha de género (sdg_05_20)

Qué mide: La diferencia porcentual entre lo que cobran hombres y mujeres. Si la brecha es del 15%, las mujeres cobran de media un 15% menos que los hombres. 0% = igualdad perfecta.

Columna: `brecha_genero`.

Dataset 6: Pobreza laboral (ilc_iw01)

Qué mide: El porcentaje de personas que TRABAJAN pero aun así están por debajo del umbral de pobreza de su país. Si es 10%, 1 de cada 10 trabajadores es pobre A PESAR de tener empleo.

Filtros elegidos: wstatus = EMP (employed, con empleo), sex = T (total, ambos sexos), age = Y18-64 (18-64 años, edad laboral).

Columna: `pobreza_laboral`.

Dataset 7: Tasa de desempleo (une_rt_a)

Qué mide: El porcentaje de personas en edad de trabajar que buscan empleo pero no lo encuentran.

Filtros: sex = T, age = Y15-74 (rango estándar de Eurostat), unit = PC_ACT (porcentaje de población activa).

Columna: `tasa_desempleo`.

Dataset 8: Inflación HICP (prc_hicp_aind)

Qué mide: El Índice Armonizado de Precios al Consumo. Mide cuánto suben los precios de los bienes básicos. Base 2015 = 100. Si HICP = 120, los precios han subido un 20% desde 2015.

Filtros: coicop = CP00 (todos los bienes, no una categoría específica), unit = I15 (índice base 2015).

Columna: `inflacion_hicp`.

Para qué lo uso: Para calcular el **salario real** = (salario nominal / HICP) × 100. Esto me dice cuánto compra realmente tu salario hoy comparado con 2015.

¿De dónde salen las decisiones de filtrado?

> **Siempre de la web de Eurostat.** Cada dataset tiene un explorador interactivo en `ec.europa.eu/eurostat/databrowser`. Al abrir un dataset, ves filtros a la izquierda con TODAS las opciones disponibles y su descripción. Así es como sé que COICOP CP00 = "All items", o que wstatus EMP = "Employed". No me lo he inventado: está documentado en la propia web de Eurostat.

¿Qué es COICOP? Son las siglas de Classification of Individual Consumption According to Purpose. Es la clasificación internacional que divide el gasto en categorías: comida (CP01), ropa (CP03), vivienda (CP04), transporte (CP07), etc. CP00 = "All items" = todos los bienes juntos, que es el índice general de inflación.

¿Qué es wstatus? Working Status. Es la dimensión del dataset de pobreza laboral que indica si la persona trabaja (EMP = employed), está en desempleo (UNE = unemployed), o está inactiva (INAC). Elegí EMP porque quiero medir pobreza entre los que TIENEN trabajo.

04. Notebook 1: limpieza y preparación

Este notebook es donde hago todo el trabajo "sucio": obtener los datos, limpiarlos, y juntarlos en un solo dataset listo para analizar. Es el 80% del trabajo real de un data scientist.

Paso 1: Cargar los datos

Cargo los 5 archivos TSV descargados de Eurostat con Pandas:

```
import pandas as pd
df_salario = pd.read_csv('./data/salario_minimo.tsv', sep='\t')
```

`sep='\t'` le dice a Pandas que el separador entre columnas es una tabulación (Tab), no una coma. Los archivos TSV de Eurostat usan tabulaciones.

Paso 2: Entender el formato raro de Eurostat

La primera columna de los TSV de Eurostat es "compuesta": junta varias dimensiones separadas por comas. Ejemplo:

```
freq,currency,geo\time    2024S2    2024S1
S,EUR,LU                  2570.93   2570.93
S,EUR,ES                  1323      1134
```

El código de país (`LU` , `ES`) está pegado a la frecuencia y la moneda. Necesito separarlo.

¿Cómo lo hago?

```
# Separo la primera columna por comas y me quedo con el último trozo
df['geo'] = df.iloc[:, 0].str.split(',').str[-1].str.strip()
```

> **¿Qué hace cada parte de esta línea?**
>
> `df.iloc[:, 0]` = "coge la primera columna del DataFrame" (iloc = integer location, [:, 0] = todas las filas, columna 0).
>
> `.str.split(',')` = "divide el texto cada vez que encuentres una coma". Convierte `"S,EUR,ES"` en `["S", "EUR", "ES"]` .
>
> `.str[-1]` = "coge el último elemento de la lista". El código de país siempre es el último.
>
> `.str.strip()` = "quita los espacios en blanco del principio y del final". Por si acaso.

Paso 3: Convertir marcadores de nulos de Eurostat

Eurostat NO usa NaN para los datos que faltan. Usa marcadores de texto como `: m` (missing), `: c` (confidencial), o `42 p` (valor provisional). Si intento hacer operaciones matemáticas con estos textos, Python explota.

```
df['valor'] = pd.to_numeric(df['valor'], errors='coerce')
```

> **¿Qué hace errors='coerce'?**
>
> `pd.to_numeric()` intenta convertir texto a número. Si el texto es `"1323"` , lo convierte a `1323.0` . Pero si es `": m"` , no puede.
>
> `errors='coerce'` le dice: "si no puedes convertirlo, pon NaN en vez de dar un error". Así todos los marcadores raros se

convierten automáticamente en NaN (valor ausente que Pandas sabe manejar).

La alternativa sería `errors='raise'` (para con error al primer texto raro) o `errors='ignore'` (deja el texto como está). Ninguna de las dos me sirve.

Paso 4: De formato ancho a largo (pd.melt)

¿Qué es formato ancho y formato largo?

Formato ancho (wide): cada año es una columna. Es como lo verías en un Excel "normal". Fácil de leer para humanos, pero difícil de analizar.

Formato largo (long): cada fila es una observación (un país en un año). Más filas, menos columnas. Difícil de leer para humanos, pero es lo que necesitan Pandas y Plotly para funcionar bien.

ANCHO (antes)			
geo	**2020**	**2021**	**2022**
ES	1108	1126	1166
FR	1539	1555	1603

↓ `pd.melt()` ↓

LARGO (después)		
geo	**anio**	**valor**
ES	2020	1108
ES	2021	1126
ES	2022	1166
FR	2020	1539
FR	2021	1555
FR	2022	1603

```
columnas_tiempo = [col for col in df.columns if col.isdigit() or 'S' in col]
df_long = pd.melt(df, id_vars=['geo'], value_vars=columnas_tiempo,
                  var_name='periodo', value_name='valor')
```

Paso 5: El problema del producto cartesiano

Este fue el error más importante del proyecto. Al hacer el primer merge (unión de tablas), me salieron 3.972 filas en vez de las ~250 esperadas. ¿Por qué? Porque dos datasets tenían VARIAS filas por cada combinación de país y año.

¿Qué es un producto cartesiano?

Imagina que tienes dos listas:

- Lista A: {manzana, pera}
- Lista B: {roja, verde, amarilla}
  El producto cartesiano es TODAS las combinaciones posibles: {manzana-roja, manzana-verde, manzana-amarilla, pera-roja, pera-verde, pera-amarilla}. Son 2 × 3 = 6 combinaciones.

  Cuando haces un merge (unión) de dos tablas y una de ellas tiene varias filas para la misma clave (ej: vivienda tenía 9 filas por país/año), Pandas combina CADA fila de una tabla con CADA fila coincidente de la otra. 9 × 2 = 18 filas donde debería haber 1.

¿Cómo lo detecté?

```
df.groupby(['geo', 'anio']).size().value_counts()
# Si devuelve algo distinto a "1    xxx", hay duplicados
```

¿Cómo lo solucioné?

Filtré ANTES del merge. Vivienda tenía categorías como TOTAL/DW_NEW/DW_EXIST × I10_A_AVG/INX_Q/etc. Me quedé solo con TOTAL + I10_A_AVG (1 fila por país/año). Gini tenía TOTAL y Y_LT18; me quedé solo con TOTAL.

> **Lección clave:** SIEMPRE verifica que cada dataset tiene exactamente 1 fila por clave (país + año) ANTES de hacer el merge. Si no lo haces, el merge genera un producto cartesiano silencioso que infla tus datos sin avisarte.

Paso 6: La API de Eurostat

El archivo TSV de salarios solo cubría ~10 periodos. Para tener la serie histórica completa desde 1999, usé la API pública de Eurostat:

```
import requests, time, random

url = 'https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/earn_mw_cur'
params = {'sinceTimePeriod': '1999-S1', 'lang': 'en'}
response = requests.get(url, params=params)
data = response.json()
```

> **¿Qué es una API?**
>
> API = Application Programming Interface. Es como un camarero en un restaurante: tú (Python) le haces un pedido (la petición HTTP) y él va a la cocina (el servidor de Eurostat), trae tu comida (los datos en formato JSON), y te la sirve. Tú nunca entras a la cocina; el camarero es el intermediario.

> **Pausa entre peticiones:** `time.sleep(random.uniform(1, 3))` entre cada llamada. Si haces muchas peticiones seguidas, el servidor te bloquea con un error 429 (Too Many Requests). La pausa aleatoria (1-3 segundos) simula un humano navegando, que es menos sospechoso que peticiones cada exactamente 1 segundo.

Paso 7: El merge final

Con los 8 datasets limpios (cada uno con exactamente 1 fila por país/año), los uní paso a paso:

```
df = salarios.merge(pib, on=['geo','anio'], how='outer')
df = df.merge(vivienda, on=['geo','anio'], how='outer')
df = df.merge(gini, on=['geo','anio'], how='outer')
# ... y así con los 8
```

¿Qué es un merge? ¿Qué es outer join?

Un **merge** es unir dos tablas por una columna común (en mi caso, `geo` + `anio`). Es como cuando en Excel usas BUSCARV para traer datos de otra tabla.

Tipos de merge:

- **Inner join:** solo conserva las filas que existen en AMBAS tablas. Si España 2003 no tiene dato de vivienda, esa fila desaparece.
- **Outer join:** conserva TODAS las filas de ambas tablas. Si falta un dato, pone NaN. No se pierde nada.
  ¿Por qué outer? Porque cada dataset cubre rangos de años diferentes. El salario va desde 1999, la vivienda desde 2015, la brecha desde 2002. Con inner perdería todas las filas donde algún dataset no tiene dato. Con outer las conservo todas y manejo los NaN hipótesis por hipótesis.

Paso 8: Ampliación a toda Europa

Inicialmente filtré solo los 27 países de la UE (EU27). Pero al ver los datos, Eurostat incluye vecinos: Noruega, Suiza, Islandia, Reino Unido, Serbia, Turquía, Montenegro, Macedonia del Norte, Albania, Bosnia y Moldavia.

Decidí incluirlos. Añadí una columna `es_eu27` (True/False) para poder colorear en los gráficos qué países son UE y cuáles no.

La narrativa: "La UE27 no existe en el vacío. Comparar con vecinos revela si el proyecto europeo protege a sus trabajadores o si el bienestar trasciende la pertenencia a la UE."

Resultado: el dataset final

Archivo: `dataset_europa_final.csv`

1.016 filas | **12 columnas** | **38 países** | **1999-2026**

Columna	Tipo	Qué es
`geo`	texto	Código de país (ES, FR, DE...)
`anio`	entero	Año
`salario_min_eur`	decimal	Salario mínimo mensual en euros
`salario_min_pps`	decimal	Salario mínimo en PPS (poder adquisitivo real)
`pib_pps`	decimal	PIB per cápita en PPS
`indice_vivienda`	decimal	Índice de precios de vivienda (2010=100)
`gini`	decimal	Desigualdad (0=igual, 100=desigual)
`brecha_genero`	decimal	Brecha salarial hombre-mujer (%)
`pobreza_laboral`	decimal	% de trabajadores que son pobres
`tasa_desempleo`	decimal	% de desempleo
`inflacion_hicp`	decimal	Índice de inflación (2015=100)
`es_eu27`	booleano	¿Es país de la UE? (True/False)

05. Notebook 2: análisis de cada hipótesis

Cada hipótesis sigue el mismo patrón:

1. **Preparar:** filtro las filas que tienen datos para las dos variables que necesito (`.dropna()`).
2. **Visualizar:** hago gráficos con Plotly para VER el patrón.
3. **Validar:** aplico el test de Pearson para MEDIR si el patrón es real.
4. **Concluir:** escribo la conclusión basándome en lo que dicen los datos, no en lo que esperaba.

Nota importante: en todas las hipótesis excepto H1 y H5 uso `salario_min_pps` (no EUR). ¿Por qué? Porque en H1 demostré que los euros engañan. Después de eso, sería incoherente usar euros.

H1: El salario nominal engaña

VALIDADA $r = 0.945$ $p < 0.001$ 21 de 27 países cambian de posición

¿Qué pregunto exactamente?

Si ordeno los países por salario en euros y luego los ordeno por salario en PPS (poder adquisitivo), ¿cambia el ranking? Si cambia mucho, el salario en euros no refleja el nivel de vida real.

¿Qué hice?

Elegí 2024 como año de referencia (el más reciente con datos de los 27 países UE). Calculé dos rankings: uno por EUR y otro por PPS. Comparé cuántas posiciones cambia cada país.

Los gráficos de la web y qué significan:

Gráfico 1: Scatter EUR vs PPS (web, sección H1)

Qué ves: Cada punto es un país. Eje horizontal = salario en euros. Eje vertical = salario en PPS. Los colores distinguen regiones: azul (Europa Occidental: LU, IE, NL, DE, BE, FR), naranja (Sur: ES, PT, EL, SI, CY, MT, HR), gris (Este: PL, RO, BG, etc.), verde (España destacada).

Cómo se lee: Si EUR y PPS fueran iguales, todos los puntos estarían en una diagonal perfecta. Los puntos POR ENCIMA de la diagonal viven mejor de lo que parece (su poder de compra es mayor que sus euros). Los de POR DEBAJO viven peor (la vida es cara y sus euros rinden menos).

Qué se ve: Los países del Este (RO, BG, PL) están por encima de la diagonal: cobran poco en euros pero la vida es barata, así que su poder de compra real es mayor de lo que sugiere el salario nominal. Irlanda (IE) está por debajo: cobra mucho en euros pero la vida es tan cara que su poder de compra real cae.

El caso de España: ES está prácticamente EN la diagonal. 1.323€ → 1.456 PPS (+10%). El coste de vida en España está cerca de la media europea, así que euros y PPS no difieren mucho.

Gráfico 2: Lollipop chart (web, dentro del análisis H1)

Qué ves: Para cada país, una línea horizontal que muestra la DISTANCIA entre lo que cobras en euros y lo que puedes comprar en PPS. A la derecha del centro: tu PPS es mayor que tus euros (vives mejor de lo que parece). A la izquierda: tu

PPS es menor (vives peor).

Colores: Verde = gana poder adquisitivo. Rojo = pierde. Amarillo = España.

Datos clave:

- Rumanía: +484 PPS sobre nominal (+61%). Mayor ganancia de toda Europa.
- Irlanda: -659 PPS respecto al nominal (-28%). El coste de vida devora el sueldo.
- Estonia: 820€ = 820 PPS. El ÚNICO país donde 1€ = 1 PPS exactamente.
- España: +138 PPS (+10%). Ligera ventaja, coste de vida algo inferior a la media.

El test de Pearson de H1

Resultado: $r = 0.945$, $p < 0.001$.

¿Qué significa? La correlación entre EUR y PPS es muy fuerte y positiva. Esto es lógico: ambas miden salario. Pero NO es 1.0 (correlación perfecta). Ese 0.055 que falta hasta 1.0 es exactamente lo que demuestra que el ajuste PPS cambia las cosas. Si fuera 1.0, EUR y PPS serían idénticos y no serviría de nada ajustar.

La prueba más contundente no es el Pearson sino el conteo: 21 de 27 países cambian de posición en el ranking. El cambio llega hasta ±8 posiciones.

Conclusión H1: Comparar salarios mínimos en euros entre países europeos es engañoso. Un mileurista en Bucarest puede tener más capacidad de compra que uno en Dublín. Para entender cuánto vale realmente un salario, hay que ajustar por el coste de vida local (PPS).

¿Por qué importa H1 para el resto del análisis? Porque después de demostrar que los euros engañan, TODAS las hipótesis siguientes (H2, H3, H4) usan PPS en vez de EUR. Sería incoherente usar euros después de probar que no reflejan la realidad.

H2: El SM como herramienta feminista

PARCIALMENTE VALIDADA $r = -0.299$ (panel) $r = -0.116$ (snapshot, $p = 0.58$)

¿Qué pregunto?

¿Los países con salario mínimo más alto tienen una brecha de género salarial más pequeña?

¿Por qué el SM podría reducir la brecha?

Las mujeres están sobrerrepresentadas en los empleos peor pagados (limpieza, hostelería, cuidados...). Cuando un gobierno sube el salario mínimo, las personas que más lo notan son las que cobraban cerca del mínimo. Y esas personas son, desproporcionadamente, mujeres. Así que subir el SM "levanta" más a las mujeres que a los hombres, reduciendo la brecha.

Scatter de H2 en la web (snapshot 2014, 25 países)

Qué ves: Cada punto es un país. Eje X = salario PPS. Eje Y = brecha de género (%). Colores por nivel de brecha: verde (<8%), amarillo (8-15%), naranja (15-20%), rojo (>20%).

Cómo se lee: Si H2 es correcta, los puntos deberían ir "hacia abajo a la derecha" (más salario = menos brecha). Se ve una tendencia ligera pero con MUCHA dispersión.

Casos extremos:

- **Estonia (EE):** punto rojo arriba a la izquierda. Salario bajo (~460 PPS) Y brecha enorme (~28%). El peor caso de Europa.
- **Luxemburgo (LU):** punto verde abajo a la derecha. Salario alto (~1.527 PPS) Y brecha baja (~5%). El mejor caso.
- **Turquía (TR):** brecha ~0%. ¿Igualdad perfecta? No. Solo el 34% de las mujeres participan en el mercado laboral. Las pocas que trabajan tienen puestos relativamente cualificados. La brecha baja es artificial.

¿Por qué hice DOS tests de Pearson?

Este es un punto sutil pero importante:

- **Test 1 (todos los años, 344 observaciones):** $r = -0.299$, $p < 0.001$. Significativo.
- **Test 2 (solo 2014, 25 observaciones):** $r = -0.116$, $p = 0.58$. NO significativo.

¿Por qué da diferente? Porque con todos los años mezclo EFECTOS TEMPORALES. Tanto la brecha de género como el salario mínimo mejoran con el tiempo (la brecha baja y el salario sube). Así que PARECEN correlacionados, pero la causa real podría ser simplemente "pasa el tiempo y ambos mejoran". El test de un solo año elimina esa confusión temporal. Si la señal desaparece (y desaparece: $p = 0.58$), parte de la correlación era temporal, no causal.

> **Conclusión H2:** A largo plazo hay señal: los países que suben el SM tienden a reducir la brecha. Pero es débil y está confundida por el tiempo. El SM es UNA herramienta para la igualdad, no LA solución. Países con SM altos pero sin políticas de igualdad complementarias (como Países Bajos con brecha del 17%) mantienen brechas significativas.

H3: El SM no da para vivir

VALIDADA (por crecimiento, no por correlación) $r = 0.044$ $p = 0.53$

¿Qué pregunto?

¿Los precios de la vivienda suben más rápido que el salario mínimo?

Esta es la hipótesis más sutil del proyecto

> **El Pearson NO da significativo** ($r = 0.044$, $p = 0.53$). Pero la hipótesis SÍ se valida. ¿Cómo es posible?

Porque **Pearson mide una cosa y la hipótesis pregunta otra**:

Lo que mide Pearson	Lo que pregunta H3
¿Los países con MÁS salario tienen vivienda MÁS cara?	¿La vivienda CRECE MÁS RÁPIDO que el salario?
Compara **niveles** (foto fija)	Compara **velocidades de crecimiento** (película)
Respuesta: No ($r = 0.044$)	Respuesta: Sí (8 de 10 países)

> **Analogía:** *Imagina dos corredores. Pearson te dice "¿el corredor que está más adelante corre más rápido?" No necesariamente: puede estar adelante porque empezó antes. La pregunta de H3 es: "¿cuál corre más rápido AHORA?". Son preguntas distintas que se miden de forma distinta.*

Barras dobles de H3 en la web (crecimiento SM vs vivienda, 2015-2024)

Qué ves: Para 10 países, dos barras por país: verde (crecimiento del salario %) y roja (crecimiento de la vivienda %). Ordenados por brecha (cuánto gana la vivienda al salario).

Datos exactos del notebook:

País	Salario	Vivienda	¿Quién gana?
Portugal	+61.5%	**+124.4%**	Vivienda (+63pp)
Países Bajos	+37.5%	**+98.2%**	Vivienda (+61pp)
Chequia	+68.5%	**+122.2%**	Vivienda (+54pp)
Irlanda	+36.4%	**+83.4%**	Vivienda (+47pp)
Alemania	+40.1%	**+47.9%**	Vivienda (+8pp)
Bulgaria	+84.5%	**+103.2%**	Vivienda (+19pp)
Polonia	+89.1%	**+106.7%**	Vivienda (+18pp)
Francia	+22.1%	**+26.4%**	Vivienda (+4pp)
España	**+82.5%**	+60.3%	SALARIO gana
Rumanía	**+167.6%**	+55.9%	SALARIO gana

Conclusión H3: En 8 de 10 países la vivienda crece más rápido que el salario. Solo España (por las subidas masivas del SM desde 2018) y Rumanía (por la convergencia salarial con Europa Occidental) son excepciones. Aunque cobres más cada año, cada año puedes comprar menos metros cuadrados.

H4: Trabajar no te saca de pobre

VALIDADA $r = -0.245$ $p < 0.001$ Media pobreza laboral: 8.6%

¿Qué pregunto?

¿Un salario mínimo bajo está asociado a más pobreza laboral (trabajadores que son pobres a pesar de trabajar)?

Pictograma "1 de cada 10" (web, inicio H4)

Qué ves: 20 figuras humanas, 2 en rojo. Representa que el 8.6% de los trabajadores europeos son pobres (casi 1 de cada 10, o exactamente 2 de cada 20).

Scatter de H4 en la web (snapshot 2017, 27 países)

Qué ves: Cada punto es un país. Eje X = salario PPS. Eje Y = pobreza laboral (%). Línea horizontal punteada roja = media UE (8.6%).

Datos clave:

- **Rumanía:** arriba a la izquierda. SM bajo (540 PPS), pobreza laboral 19.7%. El peor caso.
- **Luxemburgo:** SORPRESA. SM más alto de Europa (1.552 PPS) PERO pobreza laboral del 13.5%. ¿Por qué? Porque el coste de vida en LU es tan alto que el SM no protege contra la pobreza real.
- **Chequia:** SM bajo (593 PPS) PERO pobreza laboral más baja (3.8%). Red de protección social efectiva que compensa el SM bajo.
- **España:** 12.2%, por encima de la media. Temporalidad y parcialidad explican parte.

El gráfico inteligente: desempleo vs pobreza laboral

Hice un gráfico adicional comparando desempleo con pobreza laboral. Resultado: r = 0.287 (correlación débil). **Son problemas DIFERENTES.** Un país puede tener pleno empleo y aun así tener trabajadores pobres. El problema no es la cantidad de empleo sino la CALIDAD (contratos temporales, jornadas parciales, subempleo).

> **Conclusión H4:** 8.6% de trabajadores europeos son pobres. El SM alto reduce la pobreza laboral, pero la correlación es débil. Y desempleo y pobreza laboral son problemas distintos: no basta con crear empleo, hay que crear empleo DIGNO.

H5: La Gen Z trabaja más para vivir peor

REFUTADA (pero con matices) r = -0.060 p = 0.13

¿Qué pregunto?

¿La inflación se ha comido el salario mínimo real? ¿Los jóvenes cobran menos en poder de compra que sus padres a su edad?

Cómo calculé el salario real

```
df['salario_real'] = (df['salario_min_eur'] / df['inflacion_hicp']) * 100
```

Si gano 1.000€ y el HICP es 120 (precios han subido un 20% desde 2015), mi salario real = (1000/120)×100 = 833€ "de 2015". Mis 1.000€ de hoy compran lo mismo que 833€ compraban en 2015.

Barras de H5 en la web (crecimiento SM real, 2008-2024)

Qué ves: Barras horizontales para 13 países. Colores: púrpura (Europa del Este), gris (Occidental), verde lima (España), rojo (Turquía).

El patrón: Europa del Este ha crecido enormemente (RO +226.8%, BG +176.4%). Es la convergencia con Occidente. Europa Occidental apenas se ha movido (BE +3.8%, FR +6.7%, DE +13.5%). España: +33.2%, por encima de la media occidental gracias a las subidas desde 2018. Turquía: -97.9%, el único país con caída real (hiperinflación).

La gran conexión H3 + H5

> **Este es el hallazgo más importante del proyecto.**
>
> H5 sola dice: "el salario real sube en casi toda Europa, la hipótesis es falsa."
> H3 sola dice: "la vivienda sube más rápido que el salario."
>
> **Juntas** dicen: la Gen Z gana un poco más en términos reales (puede comprar un poco más de comida, ropa, tecnología que sus padres a su edad). PERO la vivienda sube mucho más rápido que el salario. Así que puede comprar más cosas pero tiene mucho más difícil alquilar un piso, independizarse, o formar una familia.
>
> **"La hipótesis es falsa en los números, pero cierta en la experiencia vivida."**
> **"La precariedad no se mide en euros. Se mide en metros cuadrados que ya no te puedes permitir."**

04-B. Notebook 1: walkthrough celda por celda

A continuación, CADA celda del Notebook 1 (EDA_Andoni.ipynb) con su código, su resultado, y la explicación de por qué existe.

Celda 0 (código)

```
# Importamos las librerías que vamos a necesitar
# Este bloque de imports va SIEMPRE al principio de cualquier notebook de EDA

import pandas as pd      # Para trabajar con tablas de datos (DataFrames)
import numpy as np       # Para operaciones matemáticas
import matplotlib.pyplot as plt  # Para hacer gráficas estáticas
import seaborn as sns    # Para hacer gráficas estadísticas más profesionales

# Esta línea hace que las gráficas aparezcan directamente en el notebook
%matplotlib inline

# Configuración visual básica para las gráficas
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 5)

# Para que no salgan warnings innecesarios que ensucian el output
import warnings
warnings.filterwarnings('ignore')

print("Librerías cargadas correctamente")
```

Resultado:
`Librerías cargadas correctamente`

Celda 1 (código)

```
# Cargamos los 5 datasets de Eurostat
# sep='\t' indica que el separador es una tabulación (formato TSV)
# Eurostat descarga los archivos en este formato por defecto

df_salario    = pd.read_csv('./data/salario_minimo.tsv',   sep='\t')
df_pib        = pd.read_csv('./data/pib_percapita.tsv',    sep='\t')
df_vivienda   = pd.read_csv('./data/precios_vivienda.tsv', sep='\t')
df_gini       = pd.read_csv('./data/gini.tsv',             sep='\t')
df_genero     = pd.read_csv('./data/brecha_genero.tsv',    sep='\t')

print("Los 5 datasets han sido cargados correctamente")
```

Resultado:
`Los 5 datasets han sido cargados correctamente`

Celda 2 (código)

```
# DATASET 1: Salario mínimo
# Siempre empezamos con las mismas 4 preguntas para cualquier dataset nuevo:
# 1. ¿Cuántas filas y columnas tiene?
# 2. ¿Qué tipos de datos tiene cada columna?
# 3. ¿Cómo son las primeras filas?
# 4. ¿Cuántos valores nulos hay?

print("=" * 60)
print("DATASET 1: Salario mínimo (earn_mw_cur)")
```

```
print("=" * 60)

print("\n--- Dimensiones ---")
print(f"Filas: {df_salario.shape[0]} | Columnas: {df_salario.shape[1]}")

print("\n--- Tipos de datos e información general ---")
print(df_salario.info())

print("\n--- Primeras 5 filas ---")
df_salario.head()
```

Resultado:

```
 ============================================================
DATASET 1: Salario mínimo (earn_mw_cur)
============================================================

--- Dimensiones ---
Filas: 38 | Columnas: 11

--- Tipos de datos e información general ---
<class 'pandas.DataFrame'>
RangeIndex: 38 entries, 0 to 37
Data columns (total 11 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   freq,currency,geo\TIME_PERIOD  38
```

Celda 3 (código)

```
# DATASETS 2 al 5: vistazo rápido al resto
for nombre, df in [
    ("PIB per cápita (tec00114)",        df_pib),
    ("Precios vivienda (prc_hpi_a)",      df_vivienda),
    ("Gini (ilc_di12)",                  df_gini),
    ("Brecha de género (sdg_05_20)",     df_genero)
]:
    print("=" * 60)
    print(f"DATASET: {nombre}")
    print("=" * 60)
    print(f"Filas: {df.shape[0]} | Columnas: {df.shape[1]}")
    print(f"Tipos: {df.dtypes.value_counts().to_dict()}")
    print(f"Nulos: {df.isnull().sum().sum()} valores nulos totales")
    print(f"Primera columna: {df.columns[0]}")
    print()
```

Resultado:

```
 ============================================================
DATASET: PIB per cápita (tec00114)
============================================================
Filas: 42 | Columnas: 13
Tipos: {dtype('int64'): 9, <StringDtype(storage='python', na_value=nan)>: 4}
Nulos: 0 valores nulos totales
Primera columna: freq,indic_ppp,ppp_cat18,geo\TIME_PERIOD

============================================================
DATASET: Precios vivienda (prc_hpi_a)
=====================================================
```

Celda 4 (código)

```
# LIMPIEZA - DATASET 1: Salario mínimo
# Trabajamos siempre sobre una copia para no perder el original
# Si algo sale mal, podemos volver a df_salario sin recargar el archivo
```

```
df_sal = df_salario.copy()

# PASO 1: Separar la primera columna
# La columna se llama "freq,currency,geo\TIME_PERIOD"
# Contiene valores como "S,EUR,ES" - separados por comas
# .str.split(',') divide el texto por las comas -> ['S', 'EUR', 'ES']
# .str[-1] coge el último elemento de cada lista -> 'ES'
# .str.strip() elimina espacios en blanco que pueda haber

df_sal['geo'] = df_sal.iloc[:, 0].str.split(',').str[-1].str.strip()

# Verificamos que funciona bien
print("Primeros valores de 'geo':")
print(df_sal['geo'].values[:30])
```

Resultado:

```
Primeros valores de 'geo':
['AL' 'AT' 'BE' 'BG' 'CH' 'CY' 'CZ' 'DE' 'DK' 'EE' 'EL' 'ES' 'FI' 'FR'
 'HR' 'HU' 'IE' 'IS' 'IT' 'LT' 'LU' 'LV' 'MD' 'ME' 'MK' 'MT' 'NL' 'NO'
 'PL' 'PT']
```

Celda 5 (código)

```
# PASO 2: Convertir de formato wide a long con pd.melt()
# Antes:  cada periodo de tiempo es una columna (formato wide)
# Después: cada fila es un pais en un periodo concreto (formato long)

# Primero identificamos las columnas de tiempo
# Son todas las columnas EXCEPTO la primera (que tiene los codigos comprimidos)
# y la nueva columna 'geo' que acabamos de crear
columnas_tiempo = [col for col in df_sal.columns
                   if col not in ['geo', df_sal.columns[0]]]

print("Columnas de tiempo detectadas:")
print(columnas_tiempo)
print(f"Total: {len(columnas_tiempo)} periodos")
```

Resultado:

```
Columnas de tiempo detectadas:
['2021-S2 ', '2022-S1 ', '2022-S2 ', '2023-S1 ', '2023-S2 ', '2024-S1 ', '2024-S2 ', '2025-S1 ', '2025-S2 ', '2026-S1 ']
Total: 10 periodos
```

Celda 6 (código)

```
''' Aquí arriba vemos que hay columnas que tienen espacios en blanco,
lo que puede causar problemas al convertir a formato long.
Para evitar problemas, vamos a renombrar las columnas de tiempo con
.str.strip() le aplicamos la función strip() (que elimina espacios) '''
```

Celda 7 (código)

```
# Limpiamos los espacios en los nombres de las columnas de tiempo
# .str.strip() elimina espacios al principio y al final de cada nombre
df_sal.columns = df_sal.columns.str.strip()

# Recalculamos las columnas de tiempo ya limpias
columnas_tiempo = [col for col in df_sal.columns
                   if col not in ['geo', df_sal.columns[0]]]
```

```
# PASO 2: pd.melt() - de wide a long
# id_vars: columna que se queda fija (el pais)
# value_vars: columnas que se convierten en filas (los periodos)
# var_name: nombre que le damos a la nueva columna de periodos
# value_name: nombre que le damos a la nueva columna de valores

df_sal_long = pd.melt(
    df_sal,
    id_vars='geo',
    value_vars=columnas_tiempo,
    var_name='periodo',
    value_name='salario_minimo_eur'
)

print(f"Filas antes del melt: {df_sal.shape[0]}")
print(f"Filas después del melt: {df_sal_long.shape[0]}")
print()
print("Primeras 10 filas:")
print(df_sal_long.head(10))
```

Resultado:

```
Filas antes del melt: 38
Filas después del melt: 380

Primeras 10 filas:
  geo  periodo salario_minimo_eur
0  AL  2021-S2               245
1  AT  2021-S2                : m
2  BE  2021-S2              1626
3  BG  2021-S2               332
4  CH  2021-S2                : m
5  CY  2021-S2                : m
6  CZ  2021-S2               596
7  DE  2021-S2              1602
8  DK  2021-S2                : m
9  EE  2021-S2               584
```

Celda 8 (código)

```
# PASO 3: Convertir salario_minimo_eur a número
# pd.to_numeric convierte texto a número
# errors='coerce' convierte lo que no pueda convertir en NaN
# (": m", "245 ", etc.)

df_sal_long['salario_minimo_eur'] = pd.to_numeric(
    df_sal_long['salario_minimo_eur'],
    errors='coerce'
)

# Comprobamos cuántos NaN hay ahora
nulos = df_sal_long['salario_minimo_eur'].isnull().sum()
total = len(df_sal_long)

print(f"Valores nulos: {nulos} de {total} ({round(nulos/total*100, 2)}%)")
print()
print(df_sal_long.dtypes)
print()
print(df_sal_long.head(10))
```

Resultado:

```
Valores nulos: 83 de 380 (21.84%)

geo                      str
periodo                  str
salario_minimo_eur    float64
dtype: object

  geo  periodo  salario_minimo_eur
0  AL  2021-S2               245.0
```

```
1  AT  2021-S2                NaN
2  BE  2021-S2             1626.0
3  BG  2021-S2              332.0
4  CH  2021-S2                NaN
5  CY  2021-S2                NaN
6  CZ  2021-S2              596.0
7  DE  2021-S2             1602.0
8  DK  2021-S2                NaN
9  EE  202
```

Celda 9 (código)

```
'''
Los ": m" son países sin salario mínimo (Dinamarca, Austria, Suecia, Suiza...)
y que tenemos 10 períodos para cada uno. Si son unos 8 países sin salario mínimo,
serían aproximadamente 80 nulos de 380 filas, algo así como un 20%.
Eso no es un problema, es información. Esos nulos nos dicen qué países no tienen
salario mínimo legal, que es relevante para el análisis.
'''
```

Celda 10 (código)

```
# PASO 4: Extraer el año del periodo y limpiar
# 'periodo' tiene valores como '2021-S2', '2022-S1'
# Necesitamos el año como número para poder hacer el merge con otros datasets
# .str[:4] coge los primeros 4 caracteres -> '2021'
# astype(int) lo convierte de texto a entero

df_sal_long['anio'] = df_sal_long['periodo'].str[:4].astype(int)

# PASO 5: Eliminar duplicados
df_sal_long = df_sal_long.drop_duplicates()

# PASO 6: Eliminar nulos de la columna principal
# Los paises sin salario minimo no nos sirven para el análisis
df_sal_long = df_sal_long.dropna(subset=['salario_minimo_eur'])

# Resultado final
print("Dataset de salario minimo limpio:")
print(f"Filas: {df_sal_long.shape[0]}")
print(f"Columnas: {df_sal_long.columns.tolist()}")
print()
print(df_sal_long.head(10))
```

Resultado:

```
Dataset de salario minimo limpio:
Filas: 297
Columnas: ['geo', 'periodo', 'salario_minimo_eur', 'anio']

   geo  periodo  salario_minimo_eur  anio
0   AL  2021-S2               245.0  2021
2   BE  2021-S2              1626.0  2021
3   BG  2021-S2               332.0  2021
6   CZ  2021-S2               596.0  2021
7   DE  2021-S2              1602.0  2021
9   EE  2021-S2               584.0  2021
10  EL  2021-S2               758.0  2021
11  ES  2021-S2              1108.0  2021
13  FR  2021-S2
```

Celda 11 (código)

```
# LIMPIEZA - DATASET 2: PIB per cápita
df_pib2 = df_pib.copy()

# Limpiamos espacios en nombres de columnas
df_pib2.columns = df_pib2.columns.str.strip()

# PASO 1: Extraer código de país
df_pib2['geo'] = df_pib2.iloc[:, 0].str.split(',').str[-1].str.strip()

# PASO 2: Columnas de tiempo
columnas_tiempo_pib = [col for col in df_pib2.columns
                       if col not in ['geo', df_pib2.columns[0]]]

# PASO 3: Melt
df_pib_long = pd.melt(
    df_pib2,
    id_vars='geo',
    value_vars=columnas_tiempo_pib,
    var_name='anio',
    value_name='pib_pps'
)

# PASO 4: Convertir año a entero y valor a número
df_pib_long['anio'] = pd.to_numeric(df_pib_long['anio'], errors='coerce')
df_pib_long['pib_pps'] = pd.to_numeric(df_pib_long['pib_pps'], errors='coerce')

# PASO 5: Eliminar duplicados y nulos
df_pib_long = df_pib_long.drop_duplicates()
df_pib_long = df_pib_long.dropna(subset=['pib_pps'])

print("Dataset PIB per cápita limpio:")
print(f"Filas: {df_pib_long.shape[0]}")
print(f"Columnas: {df_pib_long.columns.tolist()}")
print()
print(df_pib_long.head(8))
```

Resultado:

```
Dataset PIB per cápita limpio:
Filas: 482
Columnas: ['geo', 'anio', 'pib_pps']

  geo  anio  pib_pps
0  AL  2013     29.0
1  AT  2013    131.0
2  BA  2013     30.0
3  BE  2013    121.0
4  BG  2013     46.0
5  CH  2013    173.0
6  CY  2013     84.0
7  CZ  2013     85.0
```

Celda 12 (código)

```
# LIMPIEZA - DATASET 3: Precios de vivienda
df_viv2 = df_vivienda.copy()
df_viv2.columns = df_viv2.columns.str.strip()
df_viv2['geo'] = df_viv2.iloc[:, 0].str.split(',').str[-1].str.strip()

columnas_tiempo_viv = [col for col in df_viv2.columns
                       if col not in ['geo', df_viv2.columns[0]]]

df_viv_long = pd.melt(
    df_viv2,
    id_vars='geo',
    value_vars=columnas_tiempo_viv,
    var_name='anio',
    value_name='indice_vivienda'
```

```
)

df_viv_long['anio'] = pd.to_numeric(df_viv_long['anio'], errors='coerce')
df_viv_long['indice_vivienda'] = pd.to_numeric(df_viv_long['indice_vivienda'], errors='coerce')
df_viv_long = df_viv_long.drop_duplicates()
df_viv_long = df_viv_long.dropna(subset=['indice_vivienda'])

print(f"Vivienda limpio: {df_viv_long.shape[0]} filas | {df_viv_long.columns.tolist()}")
print(df_viv_long.head(5))
```

Resultado:

```
Vivienda limpio: 2980 filas | ['geo', 'anio', 'indice_vivienda']
  geo  anio  indice_vivienda
0  AT  2015           128.63
1  BE  2015           108.42
4  CZ  2015           105.40
5  DE  2015           119.20
7  EA  2015            99.20
```

Celda 13 (código)

```
# LIMPIEZA - DATASET 4: Gini
df_gin2 = df_gini.copy()
df_gin2.columns = df_gin2.columns.str.strip()
df_gin2['geo'] = df_gin2.iloc[:, 0].str.split(',').str[-1].str.strip()

columnas_tiempo_gin = [col for col in df_gin2.columns
                       if col not in ['geo', df_gin2.columns[0]]]

df_gin_long = pd.melt(
    df_gin2,
    id_vars='geo',
    value_vars=columnas_tiempo_gin,
    var_name='anio',
    value_name='gini'
)

df_gin_long['anio'] = pd.to_numeric(df_gin_long['anio'], errors='coerce')
df_gin_long['gini'] = pd.to_numeric(df_gin_long['gini'], errors='coerce')
df_gin_long = df_gin_long.drop_duplicates()
df_gin_long = df_gin_long.dropna(subset=['gini'])

print(f"Gini limpio: {df_gin_long.shape[0]} filas | {df_gin_long.columns.tolist()}")
print(df_gin_long.head(5))
```

Resultado:

```
Gini limpio: 639 filas | ['geo', 'anio', 'gini']
  geo  anio  gini
1  AT  2016  27.2
2  BE  2016  26.3
4  CH  2016  29.4
5  CY  2016  32.1
6  CZ  2016  25.1
```

Celda 14 (código)

```
# LIMPIEZA - DATASET 5: Brecha de género
df_gen2 = df_genero.copy()
df_gen2.columns = df_gen2.columns.str.strip()
df_gen2['geo'] = df_gen2.iloc[:, 0].str.split(',').str[-1].str.strip()

columnas_tiempo_gen = [col for col in df_gen2.columns
                       if col not in ['geo', df_gen2.columns[0]]]

df_gen_long = pd.melt(
```

```
    df_gen2,
    id_vars='geo',
    value_vars=columnas_tiempo_gen,
    var_name='anio',
    value_name='brecha_genero'
)

df_gen_long['anio'] = pd.to_numeric(df_gen_long['anio'], errors='coerce')
df_gen_long['brecha_genero'] = pd.to_numeric(df_gen_long['brecha_genero'], errors='coerce')
df_gen_long = df_gen_long.drop_duplicates()
df_gen_long = df_gen_long.dropna(subset=['brecha_genero'])

print(f"Genero limpio: {df_gen_long.shape[0]} filas | {df_gen_long.columns.tolist()}")
print(df_gen_long.head(5))
```

Resultado:

```
Genero limpio: 531 filas | ['geo', 'anio', 'brecha_genero']
   geo  anio  brecha_genero
3   BG  2002           18.9
5   CY  2002           22.5
6   CZ  2002           22.1
12  EL  2002           25.5
13  ES  2002           20.2
```

Celda 15 (código)

```
# Lista oficial de los 27 países de la Unión Europea
# Usamos los códigos ISO de 2 letras que usa Eurostat
# Es importante tenerla definida aquí para reutilizarla en todo el proyecto

EU27 = [
    'AT',  # Austria
    'BE',  # Bélgica
    'BG',  # Bulgaria
    'CY',  # Chipre
    'CZ',  # Chequia
    'DE',  # Alemania
    'DK',  # Dinamarca
    'EE',  # Estonia
    'EL',  # Grecia (Eurostat usa EL, no GR)
    'ES',  # España
    'FI',  # Finlandia
    'FR',  # Francia
    'HR',  # Croacia
    'HU',  # Hungría
    'IE',  # Irlanda
    'IT',  # Italia
    'LT',  # Lituania
    'LU',  # Luxemburgo
    'LV',  # Letonia
    'MT',  # Malta
    'NL',  # Países Bajos
    'PL',  # Polonia
    'PT',  # Portugal
    'RO',  # Rumanía
    'SE',  # Suecia
    'SI',  # Eslovenia
    'SK',  # Eslovaquia
]

print(f"Países UE27 definidos: {len(EU27)}")
print(EU27)
```

Resultado:

```
Países UE27 definidos: 27
['AT', 'BE', 'BG', 'CY', 'CZ', 'DE', 'DK', 'EE', 'EL', 'ES', 'FI', 'FR', 'HR', 'HU', 'IE', 'IT', 'LT', 'LU', 'LV', 'MT', 'NL',
'PL', 'PT', 'RO', 'SE', 'SI', 'SK']
```

Celda 16 (código)

```
# Preparamos el salario mínimo para el merge
# Necesitamos un solo valor por país y año (no dos semestres)
# Calculamos la media de S1 y S2 para cada año

df_sal_anual = df_sal_long.groupby(['geo', 'anio'])['salario_minimo_eur'].mean().reset_index()

print("Salario mínimo anualizado:")
print(f"Filas: {df_sal_anual.shape[0]}")
print(df_sal_anual[df_sal_anual['geo'] == 'ES']) # Miramos España
```

Resultado:

```
Salario mínimo anualizado:
Filas: 178
   geo  anio  salario_minimo_eur
46  ES  2021              1108.0
47  ES  2022              1146.5
48  ES  2023              1213.5
49  ES  2024              1323.0
50  ES  2025              1381.0
51  ES  2026              1381.0
```

Celda 17 (código)

```
# MERGE de los 5 datasets
# Unimos uno a uno usando geo y anio como clave
# how='outer' conserva todas las filas aunque falte algún dato
# Después filtramos a EU27 y años 2016-2024

# Merge 1: salario + PIB
df_merge = pd.merge(df_sal_anual, df_pib_long, on=['geo', 'anio'], how='outer')

# Merge 2: + vivienda
df_merge = pd.merge(df_merge, df_viv_long, on=['geo', 'anio'], how='outer')

# Merge 3: + gini
df_merge = pd.merge(df_merge, df_gin_long, on=['geo', 'anio'], how='outer')

# Merge 4: + brecha de género
df_merge = pd.merge(df_merge, df_gen_long, on=['geo', 'anio'], how='outer')

print("DataFrame antes de filtrar:")
print(f"Filas: {df_merge.shape[0]} | Columnas: {df_merge.shape[1]}")
print(f"Columnas: {df_merge.columns.tolist()}")
print()

# Filtramos a EU27 y años 2016-2024
df_final = df_merge[
    (df_merge['geo'].isin(EU27)) &
    (df_merge['anio'] >= 2016) &
    (df_merge['anio'] <= 2024)
].copy()

df_final = df_final.reset_index(drop=True)

print("DataFrame final (EU27, 2016-2024):")
print(f"Filas: {df_final.shape[0]} | Columnas: {df_final.shape[1]}")
print()
print(df_final.head(40))
```

Resultado:

```
DataFrame antes de filtrar:
Filas: 5695 | Columnas: 7
Columnas: ['geo', 'anio', 'salario_minimo_eur', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero']

DataFrame final (EU27, 2016-2024):
Filas: 3972 | Columnas: 7
```

```
   geo  anio  salario_minimo_eur  pib_pps  indice_vivienda  gini  \
0   AT  2016                 NaN    128.0           136.27  27.2
1   AT  2016                 NaN    128.0           136.27  25.9
2   AT  2016                 NaN    128.0           105.95  27.2
3   AT
```

Celda 18 (texto explicativo del notebook)

Diagnóstico del merge (por qué me ha salido mal): producto cartesiano Al hacer el merge, el DataFrame resultó con 3972 filas. Eso es demasiado. El problema es que algunos datasets tienen **varias filas por país y año** (distintas categorías de vivienda, distintos grupos en Gini...). Cuando Pandas encuentra, por ejemplo, 9 filas de vivienda y 2 filas de Gini para Austria-2016, las combina todas con todas: 9 × 2 = 18 filas. Esto se llama **producto cartesiano** (gracias IA/Claude) y es el error más común en merges con datos de Eurostat por lo que he podido leer. Antes de repetir el merge, necesitamos que cada dataset tenga **exactamente una fila por país y año**.

Celda 19 (código)

```
# Ver cuántas filas tiene cada dataset limpio antes del merge
print("Filas por dataset:")
print(f"  Salarios: {len(df_sal_anual)}")
print(f"  PIB:      {len(df_pib_long)}")
print(f"  Vivienda: {len(df_viv_long)}")
print(f"  Gini:     {len(df_gin_long)}")
print(f"  Brecha:   {len(df_gen_long)}")

print("\n--- Vivienda: categorías únicas ---")
print(df_viv_long.columns.tolist())
print(df_viv_long.head(15))

print("\n--- Gini: valores únicos por país/año ---")
print(df_gin_long.groupby(['geo', 'anio']).size().value_counts())
```

Resultado:

```
Filas por dataset:
  Salarios: 178
  PIB:      482
  Vivienda: 2980
  Gini:     639
  Brecha:   531

--- Vivienda: categorías únicas ---
['geo', 'anio', 'indice_vivienda']
          geo  anio  indice_vivienda
0          AT  2015           128.63
1          BE  2015           108.42
4          CZ  2015           105.40
5          DE  2015           119.20
7          EA  2015            99.20
8        EA19  2015            99.35
9        EA20  2015            99.29
10       EA21  2015            9
```

Celda 20 (código)

```
# ============================================================
# PASO 1: Investigar qué categorías tiene el dataset de vivienda
# ============================================================
# El dataset original de vivienda tiene una primera columna compuesta
# tipo "freq,purchase,unit,geo\time" que contiene categorías extra.
# Esas categorías (tipo de vivienda, tipo de índice) son las que
```

```
# generan varias filas por país/año.
#
# Necesitamos ver qué hay ahí para decidir con qué nos quedamos.
# ============================================================

print("=== Dataset de vivienda: primeras filas del original ===")
print(df_vivienda.iloc[:5, :5])

print("\n=== Valores únicos en la primera columna ===")
print(df_vivienda.iloc[:, 0].unique())
```

Resultado:

```
=== Dataset de vivienda: primeras filas del original ===
  freq,purchase,unit,geo\TIME_PERIOD    2015     2016     2017     2018
0             A,DW_EXST,I10_A_AVG,AT  128.63   136.27   144.48   153.57
1             A,DW_EXST,I10_A_AVG,BE  108.42   110.71   114.46   117.96
2             A,DW_EXST,I10_A_AVG,BG  92.29 b   99.07   108.70   116.35
3             A,DW_EXST,I10_A_AVG,CY  91.93 b   91.65    91.62    90.26
4             A,DW_EXST,I10_A_AVG,CZ  105.40   113.10   126.00   136.30

===
```

Celda 21 (código)

```
# Solo el diagnóstico de Gini
print("--- Gini: filas por país/año ---")
print(df_gin_long.groupby(['geo', 'anio']).size().value_counts())

print("\n--- Gini: primeras filas del original ---")
print(df_gini.iloc[:5, :5])
print("\n--- Valores únicos en primera columna de Gini ---")
print(df_gini.iloc[:, 0].unique())
```

Resultado:

```
--- Gini: filas por país/año ---
2    319
1      1
Name: count, dtype: int64

--- Gini: primeras filas del original ---
  freq,age,statinfo,geo\TIME_PERIOD   2016   2017   2018    2019
0               A,TOTAL,GINI_HND,AL      :   36.8    35.4   34.3
1               A,TOTAL,GINI_HND,AT   27.2   27.9    26.8   27.5
2               A,TOTAL,GINI_HND,BE   26.3   26.1    25.7  25.1 b
3               A,TOTAL,GINI_HND,BG  37.7 b  40.2    39.6   40.8
4               A,TOTAL,GINI_HND,CH   29.4   30.1
```

Celda 22 (código)

```
# Ver cuántas filas tiene cada dataset limpio antes del merge
print("Filas por dataset:")
print(f"  Salarios: {len(df_sal_anual)}")
print(f"  PIB:      {len(df_pib_long)}")
print(f"  Vivienda: {len(df_viv_long)}")
print(f"  Gini:     {len(df_gin_long)}")
print(f"  Brecha:   {len(df_gen_long)}")

print("\n--- Vivienda: categorías únicas ---")
print(df_viv_long.columns.tolist())
print(df_viv_long.head(15))

print("\n--- Gini: valores únicos por país/año ---")
print(df_gin_long.groupby(['geo', 'anio']).size().value_counts())
```

Resultado:

```
Filas por dataset:
  Salarios: 178
  PIB:      482
  Vivienda: 2980
  Gini:     639
  Brecha:   531

--- Vivienda: categorías únicas ---
['geo', 'anio', 'indice_vivienda']
          geo  anio  indice_vivienda
0          AT  2015           128.63
1          BE  2015           108.42
4          CZ  2015           105.40
5          DE  2015           119.20
7          EA  2015            99.20
8        EA19  2015            99.35
9        EA20  2015            99.29
10       EA21  2015            9
```

Celda 23 (texto explicativo del notebook)

Diagnóstico: ¿por qué el merge genera filas duplicadas? Al investigar los datasets originales, descubrimos que **dos de los cinco** tienen varias filas por país y año: ### Dataset de vivienda (`precios_vivienda.tsv`) La primera columna contiene valores como `A,DW_EXST,I10_A_AVG,AT`, donde: - `DW_EXST` = vivienda existente, `DW_NEW` = vivienda nueva, `TOTAL` = ambas - `I10_A_AVG` = índice con base 2010 (cuánto han subido los precios), `RCH_A_AVG` = tasa de cambio anual (% de subida respecto al año anterior) Cada combinación (tipo de vivienda × tipo de índice) genera una fila distinta. Por ejemplo, Austria 2016 tiene **9 filas** en vivienda. ### Dataset de Gini (`gini.tsv`) La primera columna contiene valores como `A,TOTAL,GINI_HND,AT` y `A,Y_LT18,GINI_HND,AT`, donde: - `TOTAL` = Gini

Celda 24 (texto explicativo del notebook)

¿Por qué elegimos estas categorías? **Vivienda: `TOTAL` + `I10_A_AVG`** - `TOTAL` porque no queremos diferenciar entre vivienda nueva y existente; nos interesa el mercado completo - `I10_A_AVG` (índice base 2010 = 100) porque nos permite comparar cuánto han subido los precios en cada país. Si España tiene 150 y Alemania tiene 120, sabemos que los precios españoles han subido un 50% desde 2010 frente al 20% de Alemania. La tasa de cambio anual (`RCH_A_AVG`) solo nos dice el % de un año a otro, que es menos útil para nuestro análisis **Gini: `TOTAL`** - El coeficiente de Gini estándar se calcula sobre toda la población. El de menores de 18 (`Y_LT18`) es un indicador específico de pobreza infantil que no necesitamos para nuestras hipótesis

Celda 25 (texto explicativo del notebook)

Relimpieza de vivienda y Gini Volvemos al dataset **original** de cada uno y repetimos la limpieza, pero esta vez extrayendo la categoría de la primera columna compuesta para poder filtrar antes del `melt()`. La estrategia es: 1. Separar la primera columna compuesta en sus partes (freq, categoría, unidad, geo) 2. Filtrar solo las filas de la categoría que nos interesa 3. Hacer el `melt()` ya con una sola fila por país/año

Celda 26 (código)

```
# ============================================================
# RELIMPIEZA DE VIVIENDA: filtrar ANTES del melt
# ============================================================
# La primera columna tiene formato: "A,DW_EXST,I10_A_AVG,AT"
#                                    freq,purchase,unit,geo
#
# Paso 1: Extraer las partes de la columna compuesta
# .str.split(',') parte el texto por las comas
# .expand=True crea una columna separada para cada parte
# ============================================================
```

```
col_compuesta = df_vivienda.columns[0]  # nombre de la primera columna

partes = df_vivienda[col_compuesta].str.split(',', expand=True)
# partes tiene ahora 4 columnas: [0]=freq, [1]=purchase, [2]=unit, [3]=geo (que es AT ej)

# Asignamos nombres claros
df_vivienda['freq'] = partes[0]
df_vivienda['purchase'] = partes[1]    # tipo de vivienda
df_vivienda['unit'] = partes[2]        # tipo de índice
df_vivienda['geo'] = partes[3]         # código de país

# Veamos qué combinaciones existen
print("Tipos de vivienda (purchase):")
print(df_vivienda['purchase'].unique())
print("\nTipos de índice (unit):")
print(df_vivienda['unit'].unique())
print("\nCombinaciones y cuántas filas tiene cada una:")
print(df_vivienda.groupby(['purchase', 'unit']).size())
```

Resultado:

```
Tipos de vivienda (purchase):
<StringArray>
['DW_EXST', 'DW_NEW', 'TOTAL']
Length: 3, dtype: str

Tipos de índice (unit):
<StringArray>
['I10_A_AVG', 'I15_A_AVG', 'RCH_A_AVG']
Length: 3, dtype: str

Combinaciones y cuántas filas tiene cada una:
purchase  unit
DW_EXST   I10_A_AVG    36
          I15_A_AVG    37
          RCH_A_AVG    37
DW_NEW    I10_A_AVG    36
          I15_A_AVG    36
          RCH_A_AVG    36
TOTAL     I10_A_AVG    37
          I15_A_AVG    37
          RCH_A_AVG    38
d
```

Celda 27 (texto explicativo del notebook)

Paso 1: Filtrar vivienda para quedarnos con una sola categoría De las 9 combinaciones posibles por país/año, nos quedamos con: - `TOTAL` (todo tipo de vivienda: nueva + segunda mano) - `I10_A_AVG` (índice con base 2010 = 100, para ver cuánto han subido los precios) Después de filtrar, eliminamos las columnas auxiliares (`freq`, `purchase`, `unit`) que ya no necesitamos, y repetimos el `melt()` para pasar de formato ancho a largo.

Celda 28 (código)

```
# ============================================================
# FILTRAR VIVIENDA: quedarnos solo con TOTAL + I10_A_AVG
# ============================================================

# Filtramos: solo las filas donde purchase es TOTAL Y unit es I10_A_AVG
# El operador & significa "las dos condiciones a la vez"
# Los paréntesis son obligatorios cuando combinas condiciones en Pandas
df_viv_filtrado = df_vivienda[
    (df_vivienda['purchase'] == 'TOTAL') &
    (df_vivienda['unit'] == 'I10_A_AVG')
].copy()
# .copy() crea una copia independiente para no modificar el original

# Verificamos: ahora debería haber ~37 filas (una por país)
print(f"Filas después de filtrar: {len(df_viv_filtrado)}")
print(f"Países únicos: {df_viv_filtrado['geo'].nunique()}")
```

```
print(f"\nPrimeras filas:")
print(df_viv_filtrado[['geo', '2016 ', '2017 ', '2018 ']].head())
```

Resultado:

```
Filas después de filtrar: 37
Países únicos: 37

Primeras filas:
    geo    2016     2017     2018
218  AT  138.00   145.02   153.68
219  BE  111.72   115.67   119.16
220  BG  101.15   109.91   117.16
221  CY   88.42    92.55    91.30
222  CZ  112.50   125.80   136.50
```

Celda 29 (código)

```
# ============================================================
# MELT DE VIVIENDA FILTRADA: de formato ancho a largo
# ============================================================
# Ahora que tenemos 1 fila por país, hacemos el melt igual que antes.
#
# Recordatorio de qué hace melt():
# Formato ANCHO:  geo | 2016 | 2017 | 2018  (años como columnas)
# Formato LARGO:  geo | anio | valor         (años como filas)
#
# id_vars = columnas que se quedan fijas (geo)
# Las demás columnas (los años) se "derriten" en dos columnas: anio y valor
# ============================================================

# Primero eliminamos las columnas auxiliares que ya no necesitamos
# axis=1 significa "eliminar columnas" (axis=0 sería filas)
columnas_auxiliares = ['freq', 'purchase', 'unit', df_vivienda.columns[0]]
df_viv_filtrado = df_viv_filtrado.drop(columns=columnas_auxiliares, errors='ignore')

# Ahora hacemos el melt: las columnas que NO son 'geo' son los años
df_viv_long = pd.melt(
    df_viv_filtrado,
    id_vars=['geo'],           # esta columna se queda fija
    var_name='anio',           # las columnas de años se convierten en filas
    value_name='indice_vivienda'  # los valores se llaman así
)

# Limpiamos: el año viene con espacios (ej: "2016 ") y el valor puede tener letras
df_viv_long['anio'] = df_viv_long['anio'].str.strip().astype(int)
df_viv_long['indice_vivienda'] = pd.to_numeric(df_viv_long['indice_vivienda'], errors='coerce')

# Verificamos
print(f"Vivienda limpia: {len(df_v
```

Resultado:

```
Vivienda limpia: 370 filas
Filas por país/año (debería ser todo 1):
1    370
Name: count, dtype: int64

Primeras filas:
    geo  anio  indice_vivienda
0    AT  2015           129.33
1    BE  2015           109.17
2    BG  2015              NaN
3    CY  2015              NaN
4    CZ  2015           105.10
5    DE  2015           119.30
6    DK  2015              NaN
7    EA  2015            98.93
8  EA19  2015            99.10
9  EA20  2015            99.05
```

Celda 30 (texto explicativo del notebook)

Paso 2: Filtrar Gini para quedarnos solo con la población total El dataset de Gini tiene 2 filas por país/año: - `TOTAL` = desigualdad de toda la población (el estándar) - `Y_LT18` = desigualdad solo entre menores de 18 años Nos quedamos con `TOTAL` y repetimos el proceso.

Celda 31 (código)

```
# ============================================================
# FILTRAR GINI: quedarnos solo con TOTAL (toda la población)
# ============================================================

# Paso 1: extraer las partes de la columna compuesta, igual que hicimos con vivienda
# Formato: "A,TOTAL,GINI_HND,AT" = freq,age,statinfo,geo
col_compuesta_gini = df_gini.columns[0]

partes_gini = df_gini[col_compuesta_gini].str.split(',', expand=True)
df_gini['freq'] = partes_gini[0]        # siempre "A" (anual)
df_gini['age'] = partes_gini[1]          # TOTAL o Y_LT18
df_gini['statinfo'] = partes_gini[2]     # siempre GINI_HND
df_gini['geo'] = partes_gini[3]          # código de país

# Paso 2: filtrar solo las filas de TOTAL
df_gin_filtrado = df_gini[df_gini['age'] == 'TOTAL'].copy()

print(f"Gini antes de filtrar: {len(df_gini)} filas")
print(f"Gini después de filtrar: {len(df_gin_filtrado)} filas")
print(f"Grupos de edad en el filtrado: {df_gin_filtrado['age'].unique()}")
```

Resultado:

```
Gini antes de filtrar: 78 filas
Gini después de filtrar: 39 filas
Grupos de edad en el filtrado: <StringArray>
['TOTAL']
Length: 1, dtype: str
```

Celda 32 (código)

```
# ============================================================
# MELT DE GINI FILTRADO
# ============================================================

# Eliminamos columnas auxiliares
columnas_aux_gini = ['freq', 'age', 'statinfo', col_compuesta_gini]
df_gin_filtrado = df_gin_filtrado.drop(columns=columnas_aux_gini, errors='ignore')

# Melt: de ancho a largo
df_gin_long = pd.melt(
    df_gin_filtrado,
    id_vars=['geo'],
    var_name='anio',
    value_name='gini'
)

# Limpieza de tipos
df_gin_long['anio'] = df_gin_long['anio'].str.strip().astype(int)
df_gin_long['gini'] = pd.to_numeric(df_gin_long['gini'], errors='coerce')

# Verificamos: ahora debería ser todo 1 fila por país/año
print(f"Gini limpio: {len(df_gin_long)} filas")
print(f"Filas por país/año (debería ser todo 1):")
print(df_gin_long.groupby(['geo', 'anio']).size().value_counts())
print(f"\nPrimeras filas:")
print(df_gin_long.head(10))
```

Resultado:

```
Gini limpio: 390 filas
Filas por país/año (debería ser todo 1):
1    390
Name: count, dtype: int64

Primeras filas:
    geo  anio  gini
0    AL  2016   NaN
1    AT  2016  27.2
2    BE  2016  26.3
3    BG  2016   NaN
4    CH  2016  29.4
5    CY  2016  32.1
6    CZ  2016  25.1
7    DE  2016  29.5
8    DK  2016  27.7
9  EA20  2016  30.7
```

Celda 33 (texto explicativo del notebook)

Paso 3: Repetir el merge Ahora que vivienda tiene 1 fila por país/año y Gini también, el merge ya no generará producto cartesiano. Usamos `outer join` para no perder ningún dato: si un país tiene dato de PIB pero no de Gini, la fila aparece con NaN en Gini (en vez de desaparecer).

Celda 34 (código)

```
# ============================================================
# MERGE DEFINITIVO
# ============================================================
# Unimos los 5 datasets paso a paso, siempre por geo + anio.
# outer join = conservar todas las filas aunque no haya coincidencia
# (las que no coinciden se rellenan con NaN)
# ============================================================

# Empezamos con salarios como base
df_merged = df_sal_anual.merge(df_pib_long, on=['geo', 'anio'], how='outer')

# Añadimos vivienda (ahora con 1 fila por país/año)
df_merged = df_merged.merge(df_viv_long, on=['geo', 'anio'], how='outer')

# Añadimos Gini (ahora con 1 fila por país/año)
df_merged = df_merged.merge(df_gin_long, on=['geo', 'anio'], how='outer')

# Añadimos brecha de género
df_merged = df_merged.merge(df_gen_long, on=['geo', 'anio'], how='outer')

print(f"DataFrame antes de filtrar:")
print(f"  Filas: {len(df_merged)} | Columnas: {len(df_merged.columns)}")
print(f"  Columnas: {df_merged.columns.tolist()}")

# ============================================================
# FILTRAR: solo países EU27 y años 2016-2024
# ============================================================

eu27 = ['AT','BE','BG','CY','CZ','DE','DK','EE','EL','ES',
        'FI','FR','HR','HU','IE','IT','LT','LU','LV','MT',
        'NL','PL','PT','RO','SE','SI','SK']

df_europa = df_merged[
    (df_merged['geo'].isin(eu27)) &
    (df_merged['anio'].between(2016, 2024))
].copy()

print(f"\nDataFrame final (EU27, 2016-
```

Resultado:

```
DataFrame antes de filtrar:
```

```
  Filas: 855 | Columnas: 7
  Columnas: ['geo', 'anio', 'salario_minimo_eur', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero']

DataFrame final (EU27, 2016-2024):
  Filas: 243 | Columnas: 7

¿Máximo de filas por país/año?: 1
(Si dice 1, el merge está bien. Si dice más de 1, sigue habiendo duplicados)

Primeras filas:
   geo  anio  salario_minimo_eur  pib_pps  indice_vivienda  gini  \
24  AT  2016                 NaN    128.0           138.00  27.2
25  AT  2017
```

Celda 35 (texto explicativo del notebook)

Estado del merge El merge funciona correctamente: **1 fila por país y año**, sin productos cartesianos. Resultado actual: **243 filas y 7 columnas** (27 países EU27 × 9 años, 2016-2024). | Requisito | Actual | Objetivo | Estado | |----------|--------|----------|--------| | Filas | 243 | 1000+ | Pendiente | | Columnas | 7 | 10+ | Pendiente | ### ¿Cómo llegamos al objetivo? **Más columnas** (necesitamos +3): - Salario mínimo en PPS (paridad de poder adquisitivo) → API de Eurostat - Salario mínimo como % del salario mediano → API de Eurostat - Potencialmente: inflación HICP → API del BCE **Más filas** (necesitamos +757): - Ampliar el rango de años desde 1999 (en vez de solo 2016) → API de Eurostat - 27 países × 25 años = hasta 675 filas adicionales ### Siguiente paso Conectar con l

Celda 36 (código)

```
# ============================================================
# GUARDAMOS EL DATAFRAME ACTUAL
# ============================================================
# Guardamos el merge actual como CSV para poder cargarlo en el
# siguiente notebook sin tener que repetir toda la limpieza.
#
# index=False evita que Pandas añada una columna extra con números
# de fila que no necesitamos.
# ============================================================

df_europa.to_csv('./data/merge_parcial_eu27.csv', index=False)
print("Guardado en ./data/merge_parcial_eu27.csv")
print(f"  {len(df_europa)} filas × {len(df_europa.columns)} columnas")
```

Resultado:

```
Guardado en ./data/merge_parcial_eu27.csv
  243 filas × 7 columnas
```

Celda 37 (texto explicativo del notebook)

Ahora vienen las APIs (que no me gustan especialmente después de la experiencia intentando conectar con ellas en varios ejercicios del Bootcamp)

Celda 38 (texto explicativo del notebook)

Conexión con la API de Eurostat El dataset de salario mínimo descargado manualmente (`salario_minimo.tsv`) solo cubre periodos recientes y una única unidad monetaria (EUR). Para el análisis necesito: - **Serie histórica desde 1999**: el salario mínimo europeo armonizado existe desde entonces, y necesito esa profundidad temporal para analizar la evolución generacional (H5) - **Salario en PPS (Paridad de Poder Adquisitivo)**: el euro nominal no refleja el coste de vida real de cada país. 1000€ en Luxemburgo no compran lo mismo que en Bulgaria. El ajuste PPS corrige esa distorsión y es fundamental para la H1 El indicador de Eurostat es `earn_mw_cur` (Minimum Wages, Current prices). La API es pública, gratuita y no requiere autenticación.

Celda 39 (código)

```
# Librerías necesarias para conectar con la API
# - requests: permite hacer peticiones HTTP a servidores web
# - time / random: para añadir pausas aleatorias entre peticiones
#   y evitar que el servidor nos bloquee por exceso de solicitudes (error 429)
# - json: para interpretar la respuesta de la API, que viene en formato JSON

import requests
import time
import random
import json

print("Librerías cargadas")
```

Resultado:

```
Librerías cargadas
```

Celda 40 (código)

```
# Petición a la API de Eurostat
# El endpoint base es fijo; el indicador earn_mw_cur se añade al final de la URL
# sinceTimePeriod filtra desde qué periodo queremos los datos

url = "https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/earn_mw_cur"

params = {
    'sinceTimePeriod': '1999-S1',
    'lang': 'en'
}

print("Solicitando datos a Eurostat...")
response = requests.get(url, params=params)
print(f"Código de respuesta: {response.status_code}")

if response.status_code == 200:
    data = response.json()
    print(f"Dimensiones disponibles: {data.get('id', [])}")
    print(f"Número total de valores: {len(data.get('value', {}))}")
else:
    print(f"Error en la petición: {response.status_code}")
    print(response.text[:500])
```

Resultado:

```
Solicitando datos a Eurostat...
Código de respuesta: 200
Dimensiones disponibles: ['freq', 'currency', 'geo', 'time']
Número total de valores: 4484
```

Celda 41 (código)

```
# Inspección del JSON: qué dimensiones contiene y qué categorías tiene cada una
# Equivalente a hacer .head() o .unique() en un DataFrame,
# pero adaptado al formato JSON-stat que usa la API de Eurostat

for dim_name in data['id']:
    dim = data['dimension'][dim_name]
    categories = list(dim['category']['index'].keys())
    print(f"\n{dim_name}: {len(categories)} categorías")
    print(f"  Ejemplos: {categories[:15]}")
    if len(categories) > 15:
        print(f"  ... y {len(categories) - 15} más")
```

Resultado:

```
freq: 1 categorías
  Ejemplos: ['S']

currency: 3 categorías
  Ejemplos: ['EUR', 'PPS', 'NAC']

geo: 39 categorías
  Ejemplos: ['BE', 'BG', 'CZ', 'DK', 'DE', 'EE', 'IE', 'EL', 'ES', 'FR', 'HR', 'IT', 'CY', 'LV', 'LT']
  ... y 24 más

time: 55 categorías
  Ejemplos: ['1999-S1', '1999-S2', '2000-S1', '2000-S2', '2001-S1', '2001-S2', '2002-S1', '2002-S2', '2003-S1', '2003-S2',
'2004-S1', '2004-S2', '2005-S1', '2005-S2', '2006-S1']
  ... y 40 más
```

Celda 42 (código)

```
# Conversión del JSON-stat a DataFrame
#
# El formato JSON-stat almacena los valores en una lista plana indexada
# numéricamente. Para reconstruir a qué combinación de dimensiones corresponde
# cada valor, se calcula la posición dentro de cada dimensión usando
# división entera (//) y módulo (%).
#
# Ejemplo simplificado con 2 monedas × 3 países × 2 periodos = 12 valores:
# Valor en posición 7 -> moneda 7//(3*2)=1, país (7%6)//2=0, periodo 7%2=1
# Es decir: moneda[1]=PPS, país[0]=AT, periodo[1]=S2

import numpy as np

dimensions = data['id']
dim_categories = {}
dim_sizes = []

for dim_name in dimensions:
    dim = data['dimension'][dim_name]
    cat_index = dim['category']['index']
    ordered_cats = sorted(cat_index.keys(), key=lambda x: cat_index[x])
    dim_categories[dim_name] = ordered_cats
    dim_sizes.append(len(ordered_cats))

# Multiplicadores para calcular la posición de cada dimensión
multipliers = []
for i in range(len(dim_sizes)):
    mult = 1
    for j in range(i + 1, len(dim_sizes)):
        mult *= dim_sizes[j]
    multipliers.append(mult)

# Reconstrucción del DataFrame fila por fila
values = data['value']
rows = []

for str_idx, val in values.items():
    idx = int(str_idx)
    row = {}
    for i, dim_name in enumerate(dimensions):
        cat_position = (idx // multipliers[i]) % dim_sizes[i]
        row[dim_name] = dim_categories[dim_name][cat_position]
    row['valor'] = val
    rows.append(row)

df_api = pd.DataFrame(rows)

print(f"DataFrame de la API:
```

Resultado:

```
DataFrame de la API: 4484 filas × 5 columnas
Columnas: ['freq', 'currency', 'geo', 'time', 'valor']
```

```
  freq currency geo     time  valor
0    S      EUR  AL  1999-S2   45.0
1    S      EUR  AL  2000-S1   47.0
2    S      EUR  AL  2000-S2   52.0
3    S      EUR  AL  2001-S1   53.0
4    S      EUR  AL  2001-S2   60.0
5    S      EUR  AL  2002-S1   63.0
6    S      EUR  AL  2002-S2   68.0
7    S      EUR  AL  2003-S1   67.0
8    S      EUR  AL  2003-S2   74.0
9    S      EUR  AL  2004-S1   75.0
```

Celda 43 (código)

```
# Inspección del DataFrame resultante de la API

for col in df_api.columns:
    if col != 'valor':
        vals = df_api[col].unique()
        print(f"\n{col}: {len(vals)} valores únicos")
        print(f"  {list(vals[:20])}")

print(f"\nRango de valores numéricos: {df_api['valor'].min():.2f} - {df_api['valor'].max():.2f}")
print(f"Nulos en valor: {df_api['valor'].isna().sum()}")
```

Resultado:

```
freq: 1 valores únicos
  ['S']

currency: 3 valores únicos
  ['EUR', 'NAC', 'PPS']

geo: 31 valores únicos
  ['AL', 'BE', 'BG', 'CY', 'CZ', 'DE', 'EE', 'EL', 'ES', 'FR', 'HR', 'HU', 'IE', 'LT', 'LU', 'LV', 'MD', 'ME', 'MK', 'MT']

time: 55 valores únicos
  ['1999-S2', '2000-S1', '2000-S2', '2001-S1', '2001-S2', '2002-S1', '2002-S2', '2003-S1', '2003-S2', '2004-S1', '2004-S2',
'2005-S1', '2005-S2', '2006-S1', '2006-S2', '2007-S1', '2007-S2', '2008-S1', '2008-S2', '2009-S1']

Rango de valores num
```

Celda 44 (código)

```
# Filtrado por unidades de interés
# EUR = euros nominales (el número "bruto" que aparece en el BOE de cada país)
# PPS = Purchasing Power Standard (paridad de poder adquisitivo)
# Se descarta NAC (moneda nacional) porque no permite comparar entre países

df_api_filtrado = df_api[df_api['currency'].isin(['EUR', 'PPS'])].copy()

# Extraer año y semestre del campo time
# El formato es "2020-S1", así que los primeros 4 caracteres son el año
# y los últimos 2 son el semestre (S1 o S2)
df_api_filtrado['anio'] = df_api_filtrado['time'].str[:4].astype(int)
df_api_filtrado['semestre'] = df_api_filtrado['time'].str[-2:]

print(f"Filas tras filtrar EUR y PPS: {len(df_api_filtrado)}")
print(f"Rango temporal: {df_api_filtrado['anio'].min()} - {df_api_filtrado['anio'].max()}")
print(f"Semestres: {df_api_filtrado['semestre'].unique()}")
print(df_api_filtrado[['geo', 'anio', 'semestre', 'currency', 'valor']].head(10))
```

Resultado:

```
Filas tras filtrar EUR y PPS: 2950
Rango temporal: 1999 - 2026
Semestres: <StringArray>
['S2', 'S1']
Length: 2, dtype: str
  geo  anio semestre currency  valor
0  AL  1999       S2      EUR   45.0
1  AL  2000       S1      EUR   47.0
2  AL  2000       S2      EUR   52.0
3  AL  2001       S1      EUR   53.0
4  AL  2001       S2      EUR   60.0
5  AL  2002       S1      EUR   63.0
6  AL  2002       S2      EUR   68.0
7  AL  2003       S1      EUR   67.0
8  AL  2003       S2      EUR   74.0
9  AL
```

Celda 45 (código)

```
# Pivotado: convertir las filas de EUR y PPS en columnas separadas
# Antes del pivot, cada combinación país/año/semestre tiene DOS filas:
#   ES | 2020 | S1 | EUR | 1108.0
#   ES | 2020 | S1 | PPS | 1050.0
#
# Después del pivot, tiene UNA fila con DOS columnas:
#   ES | 2020 | S1 | salario_min_eur=1108.0 | salario_min_pps=1050.0
#
# pivot_table() es la operación inversa a melt():
#   melt()        -> convierte columnas en filas (formato ancho a largo)
#   pivot_table() -> convierte filas en columnas (formato largo a ancho)

df_pivot = df_api_filtrado.pivot_table(
    index=['geo', 'anio', 'semestre'],  # columnas que se quedan como filas
    columns='currency',                  # los valores de currency se convierten en columnas
    values='valor',                      # los números que rellenan las nuevas columnas
    aggfunc='first'                      # si hay duplicados, se conserva el primero
).reset_index()

# Limpiar el nombre del grupo de columnas que deja pivot_table
df_pivot.columns.name = None

# Renombrar para mayor claridad
df_pivot = df_pivot.rename(columns={
    'EUR': 'salario_min_eur',
    'PPS': 'salario_min_pps'
})

print(f"Tras pivotar: {len(df_pivot)} filas × {len(df_pivot.columns)} columnas")
print(f"Columnas: {df_pivot.columns.tolist()}")
print(df_pivot.head(10))
```

Resultado:

```
Tras pivotar: 1532 filas × 5 columnas
Columnas: ['geo', 'anio', 'semestre', 'salario_min_eur', 'salario_min_pps']
  geo  anio semestre  salario_min_eur  salario_min_pps
0  AL  1999       S1              NaN             85.0
1  AL  1999       S2             45.0             93.0
2  AL  2000       S1             47.0             94.0
3  AL  2000       S2             52.0            104.0
4  AL  2001       S1             53.0            103.0
5  AL  2001       S2             60.0            112.0
6
```

Celda 46 (código)

```
# Media anual: promedio de S1 (enero) y S2 (julio) para obtener un valor por país y año
# Mismo proceso que aplicamos al dataset de salarios del TSV
#
# groupby(['geo', 'anio']) agrupa las filas que comparten país y año
# .mean(numeric_only=True) calcula la media solo de columnas numéricas
#   (numeric_only=True evita que intente calcular la media de 'semestre', que es texto)
# .reset_index() devuelve el resultado a un DataFrame normal

df_salarios_api = df_pivot.groupby(['geo', 'anio']).mean(numeric_only=True).reset_index()

# Filtrar solo los 27 países de la UE (descartar AL, MD, ME, MK, etc.)
df_salarios_api = df_salarios_api[df_salarios_api['geo'].isin(eu27)].copy()

print(f"Salarios API (EU27, media anual): {len(df_salarios_api)} filas")
print(f"Columnas: {df_salarios_api.columns.tolist()}")
print(f"Rango: {df_salarios_api['anio'].min()} - {df_salarios_api['anio'].max()}")
print(f"Países: {df_salarios_api['geo'].nunique()}")
print(f"\nPrimeras filas:")
print(df_salarios_api.head(10))
print(f"\nÚltimas filas:")
print(df_salarios_api.tail(10))
```

Resultado:

```
 Salarios API (EU27, media anual): 566 filas
Columnas: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps']
Rango: 1999 - 2026
Países: 22

Primeras filas:
   geo  anio  salario_min_eur  salario_min_pps
28  BE  1999           1085.0            960.0
29  BE  2000           1096.0           1005.0
30  BE  2001           1129.0           1031.0
31  BE  2002           1151.5           1064.5
32  BE  2003           1174.5           1072.5
33  BE  2004           1186.0           1083.0
34  BE  2005
```

Celda 47 (texto explicativo del notebook)

Merge definitivo Se rehace el merge utilizando los datos de salario de la API como base, ya que aportan: - Serie histórica desde 1999 (vs. solo periodos recientes del TSV descargado) - Dos columnas de salario: EUR (nominal) y PPS (ajustado a poder adquisitivo) Se mantiene el outer join para conservar el máximo de datos posible. Las combinaciones de país/año donde solo existe dato de salario (años 1999-2005, por ejemplo) pero no de PIB o vivienda, aparecerán con NaN en esas columnas.

Celda 48 (código)

```
# Merge definitivo: salarios API + PIB + vivienda + Gini + brecha de género
# Se usa outer join para conservar todas las filas aunque algún dataset
# no tenga dato para esa combinación de país/año

df_final = df_salarios_api.merge(df_pib_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_viv_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gin_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gen_long, on=['geo', 'anio'], how='outer')

# Filtrar EU27 (el outer join puede haber traído países de otros datasets)
df_final = df_final[df_final['geo'].isin(eu27)].copy()

# Ordenar para legibilidad
df_final = df_final.sort_values(['geo', 'anio']).reset_index(drop=True)

print("=" * 60)
print("MERGE DEFINITIVO")
print("=" * 60)
print(f"Filas: {len(df_final)}")
print(f"Columnas: {len(df_final.columns)}")
print(f"Nombres: {df_final.columns.tolist()}")
```

```
print(f"Rango temporal: {df_final['anio'].min()} - {df_final['anio'].max()}")
print(f"Países: {df_final['geo'].nunique()}")

dupes = df_final.groupby(['geo', 'anio']).size()
print(f"\nMáx filas por país/año: {dupes.max()} (debe ser 1)")

print(f"\nNulos por columna:")
print(df_final.isnull().sum())

print(f"\n% de nulos por columna:")
print((df_final.isnull().sum() / len(df_final) * 100).round(1))

print(f"\nPrimeras filas:")
print(df_final.head(15))
```

Resultado:

```
 ============================================================
MERGE DEFINITIVO
============================================================
Filas: 693
Columnas: 8
Nombres: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero']
Rango temporal: 1999 - 2026
Países: 27

Máx filas por país/año: 1 (debe ser 1)

Nulos por columna:
geo                  0
anio                 0
salario_min_eur    127
salario_min_pps    127
pib_pps            384
indice
```

Celda 49 (código)

```
# Guardar dataset definitivo

df_final.to_csv('./data/dataset_europa_final.csv', index=False)

print(f"Guardado: ./data/dataset_europa_final.csv")
print(f"  {len(df_final)} filas × {len(df_final.columns)} columnas")
print(f"\nVERIFICACIÓN DE REQUISITOS:")
print(f"  Filas:    {len(df_final)} ")
print(f"  Columnas: {len(df_final.columns)}")
```

Resultado:

```
Guardado: ./data/dataset_europa_final.csv
  693 filas × 8 columnas

VERIFICACIÓN DE REQUISITOS:
  Filas:    693
  Columnas: 8
```

Celda 50 (texto explicativo del notebook)

Datasets adicionales de Eurostat Con los 5 datasets iniciales obtengo 693 filas y 8 columnas. Para cumplir los requisitos mínimos del proyecto (1000+ filas, 10+ columnas) y fortalecer las hipótesis, descargo 3 indicadores adicionales de la API de Eurostat: | Indicador | Código Eurostat | Columna nueva | Hipótesis relacionada | |----------|----------|----------|----------| | Tasa de pobreza laboral | `ilc_iw01` | `pobreza_laboral` | H4: el empleo no previene la pobreza | | Tasa de desempleo | `une_rt_a` | `tasa_desempleo` | H4 y H5: contexto del mercado laboral | | Inflación (HICP) | `prc_hicp_aind` | `inflacion_hicp` | H5: salario real ajustado por inflación | Los tres se descargan de la API pública de Eurostat con el mismo método utilizado para el salario mín

Celda 51 (código)

```
# ============================================================
# FUNCIÓN REUTILIZABLE PARA DESCARGAR DE LA API DE EUROSTAT
# ============================================================
# En vez de repetir el mismo bloque de código 3 veces (una por cada
# indicador nuevo), creo una función que encapsula todo el proceso:
# petición HTTP -> parseo del JSON-stat -> conversión a DataFrame.
#
# Una función es un bloque de código con nombre que puedo llamar
# cuando quiera pasándole parámetros diferentes. Es como crear
# mi propia herramienta dentro de Python.
#
# def nombre(parametros):  -> define la función
#     ...código...
#     return resultado     -> devuelve el resultado
#
# Luego la llamo así: resultado = nombre(parametros)
# ============================================================

def descargar_eurostat(indicador, desde='1999-S1'):
    """
    Descarga un indicador de la API de Eurostat y lo devuelve como DataFrame.

    Parámetros:
    - indicador: código del dataset en Eurostat (ej: 'ilc_iw01')
    - desde: periodo inicial (por defecto '1999-S1')

    Devuelve: DataFrame con columnas por cada dimensión + columna 'valor'
    """

    url = f"https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/{indicador}"
    params = {'sinceTimePeriod': desde, 'lang': 'en'}

    print(f"Descargando {indicador}...")
    response = requests.get(url, params=params)

    if response.status_code != 200:
        print(f"  ERROR: código {response.status
```

Resultado:

```
Función descargar_eurostat() definida
```

Celda 52 (código)

```
# ============================================================
# DESCARGA DE LOS 3 INDICADORES ADICIONALES
# ============================================================
# Entre cada petición añado una pausa aleatoria de 1 a 3 segundos
# para no sobrecargar el servidor de Eurostat.
# random.uniform(1, 3) genera un número decimal aleatorio entre 1 y 3.
# time.sleep() pausa la ejecución ese número de segundos.
# ============================================================

# 1. Tasa de pobreza laboral (ilc_iw01)
#    Porcentaje de personas empleadas que están en riesgo de pobreza.
#    Directamente vinculado a la H4: "tener trabajo no te saca de pobre"
df_pobreza_raw = descargar_eurostat('ilc_iw01')

time.sleep(random.uniform(1, 3))  # pausa para no saturar el servidor

# 2. Tasa de desempleo anual (une_rt_a)
#    Porcentaje de población activa que no tiene empleo.
#    Contexto necesario para H4 y H5
df_desempleo_raw = descargar_eurostat('une_rt_a')

time.sleep(random.uniform(1, 3))

# 3. Inflación HICP - índice anual (prc_hicp_aind)
```

```
#    Índice Armonizado de Precios al Consumo.
#    Necesario para la H5: calcular si el salario mínimo real (descontando
#    la inflación) ha subido o bajado para la generación Z
df_inflacion_raw = descargar_eurostat('prc_hicp_aind')

print("\n" + "=" * 50)
print("Las 3 descargas se completaron")
```

Resultado:

```
Descargando ilc_iw01...
  OK: 64316 filas × 8 columnas
  Dimensiones: ['freq', 'wstatus', 'sex', 'age', 'unit', 'geo', 'time']
Descargando une_rt_a...
  OK: 39669 filas × 7 columnas
  Dimensiones: ['freq', 'age', 'unit', 'sex', 'geo', 'time']
Descargando prc_hicp_aind...
  ERROR: código 413

==================================================
Las 3 descargas se completaron
```

Celda 53 (código)

```
# ============================================================
# INSPECCIÓN DE LOS 3 DATASETS DESCARGADOS
# ============================================================
# Antes de limpiar, siempre miro qué he recibido.
# Necesito saber: qué columnas tiene cada uno, qué valores únicos
# hay en cada dimensión, y cómo se llaman las cosas.
# ============================================================

datasets_raw = {
    'Pobreza laboral': df_pobreza_raw,
    'Desempleo': df_desempleo_raw,
    'Inflación HICP': df_inflacion_raw
}

for nombre, df in datasets_raw.items():
    print(f"\n{'=' * 50}")
    print(f"{nombre}")
    print(f"{'=' * 50}")
    print(f"Filas: {len(df)} | Columnas: {df.columns.tolist()}")
    for col in df.columns:
        if col != 'valor':
            vals = df[col].unique()
            print(f"  {col}: {len(vals)} únicos -> {list(vals[:10])}")
```

Resultado:

```

==================================================
Pobreza laboral
==================================================
Filas: 64316 | Columnas: ['freq', 'wstatus', 'sex', 'age', 'unit', 'geo', 'time', 'valor']
  freq: 1 únicos -> ['A']
  wstatus: 3 únicos -> ['EMP', 'NSAL', 'SAL']
  sex: 3 únicos -> ['F', 'M', 'T']
  age: 12 únicos -> ['Y16-19', 'Y16-24', 'Y16-29', 'Y18-24', 'Y18-64', 'Y20-24', 'Y20-29', 'Y25-29', 'Y25-54', 'Y55-64']
  unit: 1 únicos -> ['PC']
  geo: 45 únicos -> ['AL', 'AT', 'C
```

Celda 54 (texto explicativo del notebook)

Descarga de inflación: petición filtrada El dataset de inflación HICP (`prc_hicp_aind`) es demasiado grande para descargarlo completo (error 413). Añado filtros directamente en la petición para pedir solo lo que necesito: índice general de precios (COICOP = CP00), índice anual (unit = INX_A_AVG), todos los países y todos los años disponibles.

Celda 55 (código)

```
# ============================================================
# DESCARGA FILTRADA DE INFLACIÓN HICP
# ============================================================
# El error 413 (Request Entity Too Large) significa que el servidor
# rechaza la petición porque el dataset completo es demasiado grande.
# Solución: añadir filtros en los parámetros de la petición para pedir
# solo la porción que necesito.
#
# COICOP es la clasificación europea de productos de consumo.
# CP00 = "All items" (todos los productos), que es el índice general
# de inflación. Sin este filtro, la API intentaría devolverme datos
# para cientos de categorías (alimentación, transporte, ropa, etc.)
# ============================================================

url_hicp = "https://ec.europa.eu/eurostat/api/dissemination/statistics/1.0/data/prc_hicp_aind"

params_hicp = {
    'sinceTimePeriod': '1999',
    'coicop': 'CP00',          # índice general (todos los productos)
    'unit': 'INX_A_AVG',       # índice anual medio (base 2015 = 100)
    'lang': 'en'
}

print("Descargando inflación HICP con filtros...")
response_hicp = requests.get(url_hicp, params=params_hicp)
print(f"Código de respuesta: {response_hicp.status_code}")

if response_hicp.status_code == 200:
    data_hicp = response_hicp.json()

    # Uso la misma lógica de conversión que la función descargar_eurostat()
    dimensions = data_hicp['id']
    dim_categories = {}
    dim_sizes = []

    for dim_name in dimensions:
        dim = data_hi
```

Resultado:

```
Descargando inflación HICP con filtros...
Código de respuesta: 200
OK: 1127 filas × 6 columnas
Columnas: ['freq', 'unit', 'coicop', 'geo', 'time', 'valor']
  freq: 1 únicos -> ['A']
  unit: 1 únicos -> ['INX_A_AVG']
  coicop: 1 únicos -> ['CP00']
  geo: 45 únicos -> ['AL', 'AT', 'BE', 'BG', 'CH', 'CY', 'CZ', 'DE', 'DK', 'EA']
  time: 27 únicos -> ['2016', '2017', '2018', '2019', '2020', '2021', '2022', '2023', '2024', '2025']
```

Celda 56 (texto explicativo del notebook)

Limpieza de los 3 datasets adicionales Cada dataset tiene múltiples categorías por país y año (sexo, edad, tipo de empleo...). Para evitar el mismo problema de producto cartesiano que tuve con vivienda y Gini, filtro cada dataset para quedarme con **una sola fila por país y año** antes del merge: - **Pobreza laboral**: `sex=T` (total, ambos sexos), `age=Y18-64` (población en edad laboral), `wstatus=EMP` (personas empleadas). Así obtengo: "de las personas empleadas de 18-64 años, ¿qué porcentaje está en riesgo de pobreza?" - **Desempleo**: `sex=T` (total), `age=Y15-74` (rango estándar de Eurostat para tasa de desempleo), `unit=PC_ACT` (porcentaje de población activa) - **Inflación**: ya viene filtrada de la descarga (CP00 + INX_A_AVG), solo extraigo país y año

Celda 57 (código)

```
# ============================================================
# LIMPIEZA DE POBREZA LABORAL (ilc_iw01)
# ============================================================
# El dataset tiene 64316 filas porque combina 3 sexos × 12 edades
# × 3 tipos de empleo × 45 países × 23 años.
# Filtro para quedarme con una sola combinación relevante:
# - wstatus = 'EMP' (todos los empleados, no solo asalariados)
# - sex = 'T' (total, ambos sexos juntos)
# - age = 'Y18-64' (población en edad laboral estándar)
# ============================================================

df_pobreza = df_pobreza_raw[
    (df_pobreza_raw['wstatus'] == 'EMP') &
    (df_pobreza_raw['sex'] == 'T') &
    (df_pobreza_raw['age'] == 'Y18-64')
].copy()

# Extraer año (viene como "2020", ya es formato año limpio)
df_pobreza['anio'] = df_pobreza['time'].astype(int)

# Renombrar geo si hace falta y quedarme solo con las columnas necesarias
df_pobreza = df_pobreza.rename(columns={'valor': 'pobreza_laboral'})
df_pobreza = df_pobreza[['geo', 'anio', 'pobreza_laboral']]

# Verificar: 1 fila por país/año
dupes = df_pobreza.groupby(['geo', 'anio']).size()
print(f"Pobreza laboral limpia: {len(df_pobreza)} filas")
print(f"Máx filas por país/año: {dupes.max()} (debe ser 1)")
print(f"Países: {df_pobreza['geo'].nunique()}")
print(f"Rango: {df_pobreza['anio'].min()} - {df_pobreza['anio'].max()}")
print(df_pobreza.head())
```

Resultado:

```
Pobreza laboral limpia: 815 filas
Máx filas por país/año: 1 (debe ser 1)
Países: 45
Rango: 2003 - 2025
      geo  anio  pobreza_laboral
21456  AL  2017             17.9
21457  AL  2018             16.6
21458  AL  2019             14.7
21459  AL  2020             12.8
21460  AL  2021             12.7
```

Celda 58 (código)

```
# ============================================================
# LIMPIEZA DE DESEMPLEO (une_rt_a)
# ============================================================
# Filtro con los valores estándar de Eurostat para tasa de desempleo:
# - sex = 'T' (total, ambos sexos)
# - age = 'Y15-74' (rango estándar para tasa de desempleo en la UE)
# - unit = 'PC_ACT' (porcentaje de la población activa)
#   Las otras opciones son PC_POP (% de toda la población, incluidos
#   niños y jubilados) y THS_PER (miles de personas, número absoluto).
#   PC_ACT es la medida estándar que sale en los titulares de prensa.
# ============================================================

df_desempleo = df_desempleo_raw[
    (df_desempleo_raw['sex'] == 'T') &
    (df_desempleo_raw['age'] == 'Y15-74') &
    (df_desempleo_raw['unit'] == 'PC_ACT')
].copy()

df_desempleo['anio'] = df_desempleo['time'].astype(int)
df_desempleo = df_desempleo.rename(columns={'valor': 'tasa_desempleo'})
df_desempleo = df_desempleo[['geo', 'anio', 'tasa_desempleo']]

dupes = df_desempleo.groupby(['geo', 'anio']).size()
print(f"Desempleo limpio: {len(df_desempleo)} filas")
print(f"Máx filas por país/año: {dupes.max()} (debe ser 1)")
print(f"Países: {df_desempleo['geo'].nunique()}")
```

```
print(f"Rango: {df_desempleo['anio'].min()} - {df_desempleo['anio'].max()}")
print(df_desempleo.head())
```

Resultado:

```
Desempleo limpio: 634 filas
Máx filas por país/año: 1 (debe ser 1)
Países: 38
Rango: 2003 - 2025
      geo  anio  tasa_desempleo
12605  AT  2009             5.7
12606  AT  2010             5.2
12607  AT  2011             4.9
12608  AT  2012             5.2
12609  AT  2013             5.7
```

Celda 59 (código)

```
# ============================================================
# LIMPIEZA DE INFLACIÓN HICP (prc_hicp_aind)
# ============================================================
# Este dataset ya viene filtrado de la descarga (CP00 + INX_A_AVG),
# así que solo necesito extraer el país, el año y el valor.
# El valor es un índice con base 2015 = 100. Si un país tiene 115
# en 2020, significa que los precios subieron un 15% desde 2015.
# ============================================================

# Identificar la columna de geo y time
print(f"Columnas de inflación: {df_inflacion_raw.columns.tolist()}")

# Identificar columna geo (puede llamarse 'geo' o 'geo\\TIME_PERIOD')
col_geo_inf = [c for c in df_inflacion_raw.columns if 'geo' in c.lower()][0]
col_time_inf = [c for c in df_inflacion_raw.columns if 'time' in c.lower()][0]

df_inflacion = df_inflacion_raw.copy()
df_inflacion = df_inflacion.rename(columns={col_geo_inf: 'geo', col_time_inf: 'time_raw'})
df_inflacion['anio'] = df_inflacion['time_raw'].astype(int)
df_inflacion = df_inflacion.rename(columns={'valor': 'inflacion_hicp'})
df_inflacion = df_inflacion[['geo', 'anio', 'inflacion_hicp']]

dupes = df_inflacion.groupby(['geo', 'anio']).size()
print(f"\nInflación limpia: {len(df_inflacion)} filas")
print(f"Máx filas por país/año: {dupes.max()} (debe ser 1)")
print(f"Países: {df_inflacion['geo'].nunique()}")
print(f"Rango: {df_inflacion['anio'].min()} - {df_inflacion['anio'].max()}")
print(df_inflacion.head())
```

Resultado:

```
Columnas de inflación: ['freq', 'unit', 'coicop', 'geo', 'time', 'valor']

Inflación limpia: 1127 filas
Máx filas por país/año: 1 (debe ser 1)
Países: 45
Rango: 1999 - 2025
  geo  anio  inflacion_hicp
0  AL  2016          101.51
1  AL  2017          104.76
2  AL  2018          106.59
3  AL  2019          108.39
4  AL  2020          110.74
```

Celda 60 (texto explicativo del notebook)

Merge definitivo con todos los datasets Ahora tengo 8 datasets limpios, cada uno con exactamente 1 fila por país/año: | Dataset | Columna(s) | Fuente | |---------|----------|-------| | Salarios API | `salario_min_eur`, `salario_min_pps` | API Eurostat `earn_mw_cur` | | PIB per cápita | `pib_pps` | TSV Eurostat `tec00114` | | Precios vivienda | `indice_vivienda` | TSV Eurostat

`prc_hpi_a` | | Coeficiente Gini | `gini` | TSV Eurostat `ilc_di12` | | Brecha de género | `brecha_genero` | TSV Eurostat `sdg_05_20` | | Pobreza laboral | `pobreza_laboral` | API Eurostat `ilc_iw01` | | Tasa de desempleo | `tasa_desempleo` | API Eurostat `une_rt_a` | | Inflación HICP | `inflacion_hicp` | API Eurostat `prc_hicp_aind` | El merge se hace con outer join sobre `geo + anio` para conservar el máximo

Celda 61 (código)

```
# ============================================================
# MERGE DEFINITIVO: 8 datasets sobre geo + anio
# ============================================================
# Empiezo con los salarios de la API como base y voy añadiendo
# cada dataset con outer join.
# Outer join = conservar todas las filas aunque no haya coincidencia.
# Los huecos se rellenan con NaN.
# ============================================================

df_final = df_salarios_api.merge(df_pib_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_viv_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gin_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gen_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_pobreza, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_desempleo, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_inflacion, on=['geo', 'anio'], how='outer')

# Filtrar solo EU27
df_final = df_final[df_final['geo'].isin(eu27)].copy()

# Ordenar
df_final = df_final.sort_values(['geo', 'anio']).reset_index(drop=True)

# Verificación completa
print("=" * 60)
print("MERGE DEFINITIVO")
print("=" * 60)
print(f"Filas: {len(df_final)}")
print(f"Columnas: {len(df_final.columns)}")
print(f"Nombres: {df_final.columns.tolist()}")
print(f"Rango temporal: {df_final['anio'].min()} - {df_final['anio'].max()}")
print(f"Países: {df_final['geo'].nunique()}")

dupes = df_final.groupby(['geo', 'anio']).size()
print(f"\nMá
```

Resultado:

```
 ============================================================
MERGE DEFINITIVO
============================================================
Filas: 751
Columnas: 11
Nombres: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero',
'pobreza_laboral', 'tasa_desempleo', 'inflacion_hicp']
Rango temporal: 1999 - 2026
Países: 27

Máx filas por país/año: 1 (debe ser 1)

Nulos por columna:
geo                  0
anio                 0
salario_min_eur
```

Celda 62 (código)

```
# ============================================================
# GUARDAR DATASET DEFINITIVO
# ============================================================
```

```
df_final.to_csv('./data/dataset_europa_final.csv', index=False)

print(f"Guardado: ./data/dataset_europa_final.csv")
print(f"  {len(df_final)} filas × {len(df_final.columns)} columnas")

print(f"\n{'=' * 50}")
print(f"VERIFICACIÓN DE REQUISITOS")
print(f"{'=' * 50}")
print(f"  Filas:    {len(df_final)} {'✓' if len(df_final) >= 1000 else '✗ (necesito 1000+)'}")
print(f"  Columnas: {len(df_final.columns)} {'✓' if len(df_final.columns) >= 10 else '✗ (necesito 10+)'}")
```

Resultado:

```
Guardado: ./data/dataset_europa_final.csv
  751 filas × 11 columnas

==================================================
VERIFICACIÓN DE REQUISITOS
==================================================
  Filas:    751 ✗ (necesito 1000+)
  Columnas: 11 ✓
```

Celda 63 (texto explicativo del notebook)

Ampliación del análisis: de EU27 a toda Europa Inicialmente el análisis se limitaba a los 27 países de la UE. Sin embargo, al construir el dataset observo que los datos de Eurostat incluyen países europeos que no son miembros de la UE: Noruega, Suiza, Islandia, Reino Unido, Turquía, Serbia y los Balcanes occidentales. Incluirlos enriquece el análisis por varias razones: - **Noruega, Suiza e Islandia** tienen algunos de los salarios más altos de Europa pero no pertenecen a la UE. ¿Su bienestar laboral depende de la pertenencia al bloque comunitario o de otros factores? - **Turquía y los Balcanes** (Serbia, Montenegro, Macedonia del Norte, Albania, Bosnia) son candidatos o potenciales candidatos a entrar en la UE. Comparar sus condiciones laborales con las de los miembros actua

Celda 64 (código)

```
# ============================================================
# DEFINIR LA LISTA AMPLIADA DE PAÍSES EUROPEOS
# ============================================================
# Amplío la lista de EU27 con los países europeos adicionales que
# aparecen en los datasets de Eurostat.
#
# Para saber qué países están disponibles, reviso los valores únicos
# de la columna 'geo' en los datasets descargados. Cada código de
# país es un estándar ISO 3166-1 alpha-2 (dos letras).
#
# Excluyo los agregados (EA20 = zona euro, EU27_2020 = media UE)
# porque no son países reales, son promedios estadísticos que
# distorsionarían el análisis.
# ============================================================

# EU27: los 27 miembros actuales de la Unión Europea
eu27 = ['AT','BE','BG','CY','CZ','DE','DK','EE','EL','ES',
        'FI','FR','HR','HU','IE','IT','LT','LU','LV','MT',
        'NL','PL','PT','RO','SE','SI','SK']

# Países europeos adicionales presentes en los datos de Eurostat
extra_europeos = [
    'NO',  # Noruega (EEE, no UE)
    'CH',  # Suiza (acuerdos bilaterales con UE, no miembro)
    'IS',  # Islandia (EEE, no UE)
    'UK',  # Reino Unido (ex-UE, salió en 2020)
    'RS',  # Serbia (candidato a UE)
    'TR',  # Turquía (candidato a UE)
    'ME',  # Montenegro (candidato a UE)
    'MK',  # Macedonia del Norte (candidato a UE)
    'AL',  # Albania (candidato a UE)
    'BA',  # Bosnia y Herzegovina (potencial candidato)
]

# Lista completa: EU27 + europeos adicionales
paises_europa = eu
```

Resultado:

```
EU27: 27 países
Extra europeos: 10 países
Total Europa: 37 países

Lista completa: ['AT', 'BE', 'BG', 'CY', 'CZ', 'DE', 'DK', 'EE', 'EL', 'ES', 'FI', 'FR', 'HR', 'HU', 'IE', 'IT', 'LT', 'LU',
'LV', 'MT', 'NL', 'PL', 'PT', 'RO', 'SE', 'SI', 'SK', 'NO', 'CH', 'IS', 'UK', 'RS', 'TR', 'ME', 'MK', 'AL', 'BA']
```

Celda 65 (código)

```
# ============================================================
# REHACER EL MERGE CON TODOS LOS PAÍSES EUROPEOS
# ============================================================
# El merge es idéntico al anterior, solo cambia el filtro final:
# en vez de filtrar por eu27, filtro por paises_europa.
# ============================================================

df_final = df_salarios_api.merge(df_pib_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_viv_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gin_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gen_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_pobreza, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_desempleo, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_inflacion, on=['geo', 'anio'], how='outer')

# Filtrar: solo países de la lista Europa (EU27 + vecinos)
df_final = df_final[df_final['geo'].isin(paises_europa)].copy()

# Ordenar por país y año
df_final = df_final.sort_values(['geo', 'anio']).reset_index(drop=True)

# Añado una columna que indica si el país es EU27 o no
# Esto será útil en el análisis para comparar miembros vs no miembros
df_final['es_eu27'] = df_final['geo'].isin(eu27)

# Verificación completa
print("=" * 60)
print("MERGE DEFINITIVO - EUROPA COMPLETA")
print("=" * 60)
print(f"Filas: {len(df_final)}")
print(f"Columnas: {len(df_final.columns)}")
print(f"Nombres: {df_final.columns.tolist()}")
print(
```

Resultado:

```
============================================================
MERGE DEFINITIVO - EUROPA COMPLETA
============================================================
Filas: 962
Columnas: 12
Nombres: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero',
'pobreza_laboral', 'tasa_desempleo', 'inflacion_hicp', 'es_eu27']
Rango temporal: 1999 - 2026
Países totales: 37
  - EU27: 27
  - Extra: 10

Máx filas por país/año: 1 (debe ser 1)

Nulos por columna:
g
```

Celda 66 (código)

```
# ============================================================
# GUARDAR DATASET DEFINITIVO
# ============================================================

df_final.to_csv('./data/dataset_europa_final.csv', index=False)

print(f"Guardado: ./data/dataset_europa_final.csv")
print(f"  {len(df_final)} filas × {len(df_final.columns)} columnas")

print(f"\n{'=' * 50}")
print(f"VERIFICACIÓN DE REQUISITOS")
print(f"{'=' * 50}")
print(f"  Filas:    {len(df_final)} {'✓' if len(df_final) >= 1000 else '✗ (necesito 1000+)'}")
print(f"  Columnas: {len(df_final.columns)} {'✓' if len(df_final.columns) >= 10 else '✗ (necesito 10+)'}")
```

Resultado:

```
Guardado: ./data/dataset_europa_final.csv
  962 filas × 12 columnas

==================================================
VERIFICACIÓN DE REQUISITOS
==================================================
  Filas:    962 ✗ (necesito 1000+)
  Columnas: 12 ✓
```

Celda 67 (texto explicativo del notebook)

Ajuste: añadir Moldova a la lista Con 37 países obtengo 962 filas, cerca del objetivo de 1000 pero sin alcanzarlo. Al revisar los valores únicos de la columna `geo` en los datasets descargados, compruebo que Moldova (MD) aparece en los datos de Eurostat pero no la había incluido en la lista inicial. Moldova es candidata oficial a la adhesión a la UE desde junio de 2022 y firmó un acuerdo de asociación con la UE en 2014. Incluirla encaja con la narrativa del análisis: comparar las condiciones laborales del bloque comunitario con las de los países que aspiran a formar parte de él.

Celda 68 (código)

```
# ============================================================
# LISTA AMPLIADA DE PAÍSES EUROPEOS
# ============================================================
# Añado MD (Moldavia) que también aparece en los datos de Eurostat.
# Moldavia firmó un acuerdo de asociación con la UE en 2014 y es
# candidato oficial a la adhesión desde 2022. Encaja con la narrativa
# de comparar el bloque comunitario con sus vecinos y aspirantes.
# ============================================================

eu27 = ['AT','BE','BG','CY','CZ','DE','DK','EE','EL','ES',
        'FI','FR','HR','HU','IE','IT','LT','LU','LV','MT',
        'NL','PL','PT','RO','SE','SI','SK']

extra_europeos = [
    'NO',  # Noruega (EEE, no UE)
    'CH',  # Suiza (acuerdos bilaterales con UE, no miembro)
    'IS',  # Islandia (EEE, no UE)
    'UK',  # Reino Unido (ex-UE, salió en 2020)
    'RS',  # Serbia (candidato a UE)
    'TR',  # Turquía (candidato a UE)
    'ME',  # Montenegro (candidato a UE)
    'MK',  # Macedonia del Norte (candidato a UE)
    'AL',  # Albania (candidato a UE)
    'BA',  # Bosnia y Herzegovina (potencial candidato)
    'MD',  # Moldavia (candidato a UE desde 2022)
]

paises_europa = eu27 + extra_europeos

print(f"EU27: {len(eu27)} países")
print(f"Extra europeos: {len(extra_europeos)} países")
print(f"Total Europa: {len(paises_europa)} países")
```

Resultado:

```
EU27: 27 países
Extra europeos: 11 países
Total Europa: 38 países
```

Celda 69 (código)

```
# ============================================================
# MERGE DEFINITIVO - EUROPA COMPLETA (con Moldova)
# ============================================================

df_final = df_salarios_api.merge(df_pib_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_viv_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gin_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gen_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_pobreza, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_desempleo, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_inflacion, on=['geo', 'anio'], how='outer')

df_final = df_final[df_final['geo'].isin(paises_europa)].copy()
df_final = df_final.sort_values(['geo', 'anio']).reset_index(drop=True)
df_final['es_eu27'] = df_final['geo'].isin(eu27)

print("=" * 60)
print("MERGE DEFINITIVO - EUROPA COMPLETA")
print("=" * 60)
print(f"Filas: {len(df_final)}")
print(f"Columnas: {len(df_final.columns)}")
print(f"Nombres: {df_final.columns.tolist()}")
print(f"Rango temporal: {df_final['anio'].min()} - {df_final['anio'].max()}")
print(f"Países totales: {df_final['geo'].nunique()}")
print(f"  - EU27: {df_final[df_final['es_eu27']]['geo'].nunique()}")
print(f"  - Extra: {df_final[~df_final['es_eu27']]['geo'].nunique()}")

dupes = df_final.groupby(['geo', 'anio']).size()
print(f"\nMáx filas por país/año: {dupes.max()} (debe ser 1)")

print(f"\nNulos por columna:")
print(
```

Resultado:

```
============================================================
MERGE DEFINITIVO - EUROPA COMPLETA
============================================================
Filas: 962
Columnas: 12
Nombres: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero',
'pobreza_laboral', 'tasa_desempleo', 'inflacion_hicp', 'es_eu27']
Rango temporal: 1999 - 2026
Países totales: 37
  - EU27: 27
  - Extra: 10

Máx filas por país/año: 1 (debe ser 1)

Nulos por columna:
g
```

Celda 70 (código)

```
# ============================================================
# GUARDAR DATASET DEFINITIVO
# ============================================================

df_final.to_csv('./data/dataset_europa_final.csv', index=False)
```

```
print(f"Guardado: ./data/dataset_europa_final.csv")
print(f"  {len(df_final)} filas × {len(df_final.columns)} columnas")

print(f"\n{'=' * 50}")
print(f"VERIFICACIÓN DE REQUISITOS")
print(f"{'=' * 50}")
print(f"  Filas:    {len(df_final)} {'✓' if len(df_final) >= 1000 else '✗ (necesito 1000+)'}")
print(f"  Columnas: {len(df_final.columns)} {'✓' if len(df_final.columns) >= 10 else '✗ (necesito 10+)'}")
```

Resultado:

```
Guardado: ./data/dataset_europa_final.csv
  962 filas × 12 columnas

==================================================
VERIFICACIÓN DE REQUISITOS
==================================================
  Filas:    962 ✗ (necesito 1000+)
  Columnas: 12 ✓
```

Celda 71 (texto explicativo del notebook)

Corrección: aplicar la lista ampliada a todos los datasets El merge sigue dando 37 países porque cuando limpié los salarios de la API, filtré por la lista `eu27` (27 países). Los países extra como Moldova, Noruega o Suiza quedaron descartados antes del merge. Para solucionarlo, vuelvo al DataFrame de salarios antes de filtrar (`df_pivot`) y rehago el cálculo de la media anual usando la lista ampliada `paises_europa`.

Celda 72 (código)

```
# ============================================================
# REHACER EL FILTRADO DE SALARIOS CON LA LISTA AMPLIADA
# ============================================================
# Cuando calculé la media anual de salarios, filtré por eu27.
# Ahora necesito filtrar por paises_europa para incluir los países
# adicionales (Noruega, Suiza, Moldova, etc.)
# Vuelvo al df_pivot (que tiene todos los países de la API) y rehago
# el cálculo con la lista ampliada.
# ============================================================

df_salarios_api = df_pivot.groupby(['geo', 'anio']).mean(numeric_only=True).reset_index()
df_salarios_api = df_salarios_api[df_salarios_api['geo'].isin(paises_europa)].copy()

print(f"Salarios API (Europa ampliada): {len(df_salarios_api)} filas")
print(f"Países: {sorted(df_salarios_api['geo'].unique())}")
print(f"¿Está Moldova (MD)?: {'MD' in df_salarios_api['geo'].values}")
```

Resultado:

```
Salarios API (Europa ampliada): 728 filas
Países: ['AL', 'BE', 'BG', 'CY', 'CZ', 'DE', 'EE', 'EL', 'ES', 'FR', 'HR', 'HU', 'IE', 'LT', 'LU', 'LV', 'MD', 'ME', 'MK',
'MT', 'NL', 'PL', 'PT', 'RO', 'RS', 'SI', 'SK', 'TR', 'UK']
¿Está Moldova (MD)?: True
```

Celda 73 (código)

```
# ============================================================
# MERGE DEFINITIVO - EUROPA COMPLETA (con lista ampliada corregida)
# ============================================================

df_final = df_salarios_api.merge(df_pib_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_viv_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_gin_long, on=['geo', 'anio'], how='outer')
```

```
df_final = df_final.merge(df_gen_long, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_pobreza, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_desempleo, on=['geo', 'anio'], how='outer')
df_final = df_final.merge(df_inflacion, on=['geo', 'anio'], how='outer')

df_final = df_final[df_final['geo'].isin(paises_europa)].copy()
df_final = df_final.sort_values(['geo', 'anio']).reset_index(drop=True)
df_final['es_eu27'] = df_final['geo'].isin(eu27)

print("=" * 60)
print("MERGE DEFINITIVO - EUROPA COMPLETA")
print("=" * 60)
print(f"Filas: {len(df_final)}")
print(f"Columnas: {len(df_final.columns)}")
print(f"Rango temporal: {df_final['anio'].min()} - {df_final['anio'].max()}")
print(f"Países totales: {df_final['geo'].nunique()}")
print(f"  - EU27: {df_final[df_final['es_eu27']]['geo'].nunique()}")
print(f"  - Extra: {df_final[~df_final['es_eu27']]['geo'].nunique()}")

dupes = df_final.groupby(['geo', 'anio']).size()
print(f"\nMáx filas por país/año: {dupes.max()} (debe ser 1)")

print(f"\nNulos por columna:")
print(df_final.isnull().sum())
```

Resultado:

```
============================================================
MERGE DEFINITIVO - EUROPA COMPLETA
============================================================
Filas: 1016
Columnas: 12
Rango temporal: 1999 - 2026
Países totales: 38
  - EU27: 27
  - Extra: 11

Máx filas por país/año: 1 (debe ser 1)

Nulos por columna:
geo                  0
anio                 0
salario_min_eur    288
salario_min_pps    319
pib_pps            592
indice_vivienda    738
gini               713
brecha_genero      521
```

Celda 74 (código)

```
# ============================================================
# GUARDAR DATASET DEFINITIVO
# ============================================================

df_final.to_csv('./data/dataset_europa_final.csv', index=False)

print(f"Guardado: ./data/dataset_europa_final.csv")
print(f"  {len(df_final)} filas × {len(df_final.columns)} columnas")

print(f"\n{'=' * 50}")
print(f"VERIFICACIÓN DE REQUISITOS")
print(f"{'=' * 50}")
print(f"  Filas:    {len(df_final)} {'✓' if len(df_final) >= 1000 else '✗ (necesito 1000+)'}")
print(f"  Columnas: {len(df_final.columns)} {'✓' if len(df_final.columns) >= 10 else '✗ (necesito 10+)'}")
```

Resultado:

```
Guardado: ./data/dataset_europa_final.csv
  1016 filas × 12 columnas
```

```
==================================================
VERIFICACIÓN DE REQUISITOS
==================================================
  Filas:    1016 ✓
  Columnas: 12 ✓
```

Celda 75 (texto explicativo del notebook)

Resumen del Notebook 1: Obtención y limpieza de datos ### Fuentes de datos - **5 datasets TSV** descargados manualmente de la web de Eurostat (salario mínimo, PIB per cápita, precios de vivienda, coeficiente Gini, brecha de género) - **3 datasets adicionales** descargados de la API pública de Eurostat (pobreza laboral, tasa de desempleo, inflación HICP) ### Proceso de limpieza 1. Separación de la columna compuesta de Eurostat (freq, categoría, unidad, geo) 2. Filtrado de categorías para evitar productos cartesianos en el merge (vivienda: TOTAL + I10_A_AVG; Gini: TOTAL; pobreza laboral: EMP + T + Y18-64; desempleo: T + Y15-74 + PC_ACT) 3. Conversión de formato ancho a largo con `pd.melt()` 4. Limpieza de tipos con `pd.to_numeric(errors='coerce')` para los marcadores de Eurostat (": m",

05-B. Notebook 2: walkthrough celda por celda

CADA celda del Notebook 2 (EDA_Hipotesis.ipynb) con su código, su resultado, y la explicación.

Celda 0 (texto explicativo del notebook)

¿Cuánto vale tu trabajo? ## Análisis exploratorio del salario mínimo y la desigualdad en Europa ### Contexto del proyecto Este proyecto analiza la evolución del salario mínimo en Europa y su relación con indicadores clave de desigualdad: poder adquisitivo, brecha de género, acceso a la vivienda, pobreza laboral e inflación. El análisis no se limita a la UE27. Incluye 38 países europeos (27 de la UE + 11 vecinos: Noruega, Suiza, Islandia, Reino Unido, Serbia, Turquía, Montenegro, Macedonia del Norte, Albania, Bosnia y Moldavia). La UE27 no existe en el vacío: comparar con vecinos revela si el proyecto europeo protege a sus trabajadores o si el bienestar trasciende la pertenencia a la UE. **Fuente de datos:** Eurostat (8 datasets: 5 descargados en TSV + 3 vía API pública). **Dataset final:** 1016 filas × 12 columnas, 38 países, periodo 1999-2026. Generado en el Notebook 1 (limpieza y merge). ### Las 5 hipótesis | # | Hipótesis | Pregunta | Variables principales | |---|----------

Celda 1 (código)

```
# ============================================================
# NOTEBOOK 2: ANÁLISIS DE HIPÓTESIS
# ============================================================
# Este notebook parte del dataset limpio que generé en el Notebook 1.
# Aquí cargo el CSV final y analizo las 5 hipótesis del proyecto,
# usando visualizaciones y tests estadísticos para validarlas.
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats  # Contiene la función pearsonr para medir correlaciones
import plotly.express as px   # Para gráficos interactivos (pasar el ratón = ver datos)

# Las gráficas aparecen directamente en el notebook
%matplotlib inline

# Estilo visual: fondo blanco con rejilla suave
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

# Desactivo warnings para que el output esté limpio
import warnings
warnings.filterwarnings('ignore')

print("Librerías cargadas correctamente")
```

Resultado:

```
Librerías cargadas correctamente
```

Celda 2 (código)

```
# ============================================================
# CARGAR EL DATASET LIMPIO DEL NOTEBOOK 1
# ============================================================
# En el Notebook 1 limpié y mergée 8 datasets de Eurostat y
# guardé el resultado en dataset_europa_final.csv.
# Aquí lo cargo directamente para no repetir todo ese proceso.
# pd.read_csv() lee un archivo CSV y lo convierte en DataFrame.
# ============================================================

df = pd.read_csv('./data/dataset_europa_final.csv')
```

```
print(f"Dataset cargado: {len(df)} filas × {len(df.columns)} columnas")
print(f"Columnas: {df.columns.tolist()}")
print(f"Rango temporal: {df['anio'].min()} - {df['anio'].max()}")
print(f"Países: {df['geo'].nunique()}")
```

Resultado:

```
Dataset cargado: 1016 filas × 12 columnas
Columnas: ['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'pib_pps', 'indice_vivienda', 'gini', 'brecha_genero',
'pobreza_laboral', 'tasa_desempleo', 'inflacion_hicp', 'es_eu27']
Rango temporal: 1999 - 2026
Países: 38
```

Celda 3 (código)

```
# ============================================================
# EXPLORACIÓN RÁPIDA
# ============================================================
# Cada vez que cargo un CSV en un notebook nuevo, lo primero
# es verificar que los tipos de datos son correctos.
# Si 'anio' se cargó como texto en vez de número, me reventaría
# una gráfica más adelante. Mejor pillarlo aquí.
# ============================================================

print("Tipos de datos:")
print(df.dtypes)
print(f"\nPrimeras filas:")
df.head(35)
```

Resultado:

```
Tipos de datos:
geo                    str
anio                 int64
salario_min_eur    float64
salario_min_pps    float64
pib_pps            float64
indice_vivienda    float64
gini               float64
brecha_genero      float64
pobreza_laboral    float64
tasa_desempleo     float64
inflacion_hicp     float64
es_eu27               bool
dtype: object

Primeras filas:

   geo  anio  salario_min_eur  salario_min_pps  pib_pps  indice_vivienda  \
0   AL  1999             45.0             89.0      NaN              NaN
1   AL  2000             49.5             99.0      NaN              NaN
```

Celda 4 (código)

```
# ============================================================
# RESUMEN ESTADÍSTICO
# ============================================================
# .describe() es como un chequeo médico del dataset.
# Para cada columna numérica calcula:
#   count = cuántos valores NO son NaN (me dice cuántos datos tengo)
#   mean  = la media (promedio)
#   std   = desviación estándar (cuánto se dispersan los datos)
#   min   = valor mínimo
#   25%   = primer cuartil (el 25% de los datos están por debajo)
#   50%   = mediana (el valor del medio, más robusto que la media)
#   75%   = tercer cuartil
```

```
#   max   = valor máximo
#
# Si veo algo raro (un salario negativo, un Gini de 200), sé que
# hay un error de limpieza que tengo que arreglar.
# ============================================================

df.describe().round(2)
```

Resultado:

```
           anio  salario_min_eur  salario_min_pps  pib_pps  indice_vivienda  \
count  1016.00           728.00           697.00   424.00           278.00
mean   2012.69           689.88           773.87    96.63           146.28
std       7.96           558.23           422.70    47.08
```

Celda 5 (código)

```
# ============================================================
# VISUALIZACIÓN DE NULOS
# ============================================================
# Un heatmap (mapa de calor) me permite ver de un vistazo dónde
# están los huecos en mis datos.
#
# ¿Qué es un heatmap?
# Es una tabla donde cada celda tiene un color en vez de un número.
# Aquí: amarillo = hay dato, morado = falta dato (NaN).
#
# ¿Por qué me importan los nulos?
# Porque cada hipótesis usa columnas distintas. Si la columna
# 'brecha_genero' tiene un 50% de NaN, solo puedo analizar H2
# con la mitad de los datos. Necesito saber esto ANTES de empezar.
#
# sns.heatmap() dibuja el mapa de calor
# df.isnull() convierte cada celda en True (NaN) o False (tiene dato)
# cmap='viridis' es la paleta de colores (amarillo-verde-morado)
# yticklabels=False quita las etiquetas del eje Y (con 1016 filas
# serían ilegibles y ralentizarían el notebook)
# ============================================================

plt.figure(figsize=(14, 8))
sns.heatmap(df.isnull(), cbar=True, yticklabels=False, cmap='viridis')
plt.title('Mapa de valores nulos en el dataset', fontsize=14)
plt.xlabel('Columnas')
plt.ylabel('Filas (cada fila = 1 país en 1 año)')
plt.tight_layout()
plt.show()

# Resumen numérico: el gráfico es para presentar, las cifras para ser preciso
print("\nNulos por columna:")
nulos = df.isnull().sum()
pct_nulos = (nulos / len(df) * 100).round(1)
resumen_nulos = pd.DataFrame({'nulos': nulos, '% del total': pct_nulos})
print(resumen_nulos[resumen_nulos['nulos'] > 0].sort_values('% del total', ascending=False))
```

Resultado:

```
[GRÁFICO GENERADO]

Nulos por columna:
                 nulos  % del total
indice_vivienda    738         72.6
gini               713         70.2
pib_pps            592         58.3
brecha_genero      521         51.3
tasa_desempleo     433         42.6
salario_min_pps    319         31.4
pobreza_laboral    312         30.7
salario_min_eur    288         28.3
inflacion_hicp     102         10.0
```

Celda 6 (código)

```
# ============================================================
# CONFIGURACIÓN PARA EL ANÁLISIS
# ============================================================
# Defino las listas de países (las mismas del Notebook 1) y una
# función reutilizable para el test de Pearson.
#
# ¿Qué es una función?
# Es un bloque de código con nombre que puedo llamar cuando quiera.
# En vez de escribir las mismas 15 líneas cada vez que quiero
# hacer un test de Pearson, las escribo UNA vez dentro de la
# función y luego la llamo con una sola línea.
# Es como crear mi propia herramienta dentro de Python.
#
# La sintaxis es:
#   def nombre_funcion(parametros):
#       ...código...
#       return resultado
#
# Y la llamo así:
#   resultado = nombre_funcion(mis_datos)
# ============================================================

eu27 = ['AT','BE','BG','CY','CZ','DE','DK','EE','EL','ES',
        'FI','FR','HR','HU','IE','IT','LT','LU','LV','MT',
        'NL','PL','PT','RO','SE','SI','SK']

extra_europeos = ['NO','CH','IS','UK','RS','TR','ME','MK','AL','BA','MD']

paises_europa = eu27 + extra_europeos


def test_pearson(df_input, col_x, col_y, nombre_test=""):
    """
    Realiza el test de correlación de Pearson entre dos columnas.

    Recibe:
    - df_input: el DataFrame con los datos
    - col_x: nombre de la primera columna (ej: 'salario_min_eur')
    - col_y: nombre de la segunda columna (ej: 'gini')
    - nombre_test: un texto descriptivo para identificar el resultado

    Devuelve: (r, p_value)
    - r: coeficiente de correlación (-1 a +1)
    - p_value: probabilidad de que sea casualidad (si < 0.05, es significativa)
    """
    # dropna() elimina las filas donde alguna de las dos columnas es NaN.
    # Pearson necesita que AMBOS valores existan en cada fila.
    # Si un país tiene salario pero no Gini, esa fila no puede participar.
    datos = df_input[[col_x, col_y]].dropna()

    # Protección: si quedan menos de 3 filas, Pearson no tiene sentido
    if len(datos) < 3
```

Resultado:

```
EU27: 27 países
Europa total: 38 países
Función test_pearson() definida
```

Celda 7 (texto explicativo del notebook)

H1: El salario nominal engaña **Hipótesis:** El salario mínimo en euros no refleja el nivel de vida real. Dos países pueden tener el mismo salario nominal pero capacidades de compra muy diferentes. El ajuste por Poder de Paridad de Compra (PPS) revela las diferencias reales. **¿Qué es PPS?** Purchasing Power Standard. Es una moneda artificial que ajusta los precios entre países. Si un café cuesta 1€ en España pero 3€ en Noruega, ganar 1000€ en España equivale a más poder de compra que 1000€ en Noruega. PPS corrige eso. **Columnas que uso:** `salario_min_eur` (nominal, en euros) y `salario_min_pps` (ajustado por poder adquisitivo) **¿Cómo la valido?** Si los rankings de países cambian significativamente al pasar de EUR a PPS, la hipótesis se confirma: el salario nominal engaña.

Celda 8 (código)

```
# ============================================================
# H1: PREPARAR LOS DATOS
# ============================================================
# Para comparar salario nominal vs PPS necesito un año concreto
# donde haya datos de ambas columnas para el máximo de países.
#
# ¿Por qué un año concreto?
# Porque quiero hacer un ranking de países. Si mezclo todos los
# años, cada país aparecería muchas veces y el ranking no tendría
# sentido. Elijo el año más reciente con datos completos.
#
# Primero miro qué años tienen más datos disponibles.
# ============================================================

# Filtro solo filas donde tengo AMBOS salarios (EUR y PPS)
df_h1 = df[['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'es_eu27']].dropna()

print("Filas con ambos salarios disponibles:")
print(f"  Total: {len(df_h1)}")
print(f"\nDatos por año:")
print(df_h1.groupby('anio')['geo'].nunique().to_string())
print(f"\n¿Qué año tiene más países? Elijo el más reciente con buena cobertura.")
```

Resultado:

```
Filas con ambos salarios disponibles:
  Total: 697

Datos por año:
anio
1999    21
2000    23
2001    23
2002    23
2003    23
2004    23
2005    23
2006    23
2007    23
2008    24
2009    24
2010    24
2011    24
2012    24
2013    26
2014    26
2015    27
2016    27
2017    27
2018    27
2019    27
2020    27
2021    26
2022    26
2023    27
2024    27
2025    26
2026    26

¿Qué año tiene más países? Elijo el más reciente con buena cobertura.
```

Celda 9 (código)

```
# ============================================================
# H1: RANKINGS EUR vs PPS
# ============================================================
```

```
# Elijo 2024 como año de referencia (el más reciente con datos).
# Creo dos rankings: uno por salario en euros y otro por salario
# en PPS. Si las posiciones cambian mucho, H1 se confirma.
#
# .sort_values() ordena el DataFrame por una columna
# .rank(ascending=False) asigna posiciones: el valor más alto = 1
# ============================================================

anio_ref = 2024
df_ranking = df_h1[df_h1['anio'] == anio_ref].copy()

# Creo las columnas de ranking
# ascending=False = el salario más ALTO recibe la posición 1
df_ranking['rank_eur'] = df_ranking['salario_min_eur'].rank(ascending=False).astype(int)
df_ranking['rank_pps'] = df_ranking['salario_min_pps'].rank(ascending=False).astype(int)

# Calculo cuántas posiciones cambia cada país al ajustar por PPS
# Un cambio positivo = sube posiciones al ajustar (vive mejor de lo que parece)
# Un cambio negativo = baja posiciones al ajustar (vive peor de lo que parece)
df_ranking['cambio_posicion'] = df_ranking['rank_eur'] - df_ranking['rank_pps']

# Ordeno por salario nominal para ver el ranking EUR
df_ranking = df_ranking.sort_values('rank_eur')

print(f"Rankings de salario mínimo en {anio_ref}")
print(f"Países con datos: {len(df_ranking)}")
print()
print(df_ranking[['geo', 'salario_min_eur', 'rank_eur', 'salario_min_pps', 'rank_pps', 'cambio_posicion']].to_string(
```

Resultado:

```
Rankings de salario mínimo en 2024
Países con datos: 27
```

geo	salario_min_eur	rank_eur	salario_min_pps	rank_pps	cambio_posicion
LU	2571.0	1	1935.0	1	0
IE	2146.0	2	1554.0	6	-4
NL	2102.0	3	1812.5	3	0
DE	2054.0	4	1891.0	2	2
BE	2032.0	5	1743.5	4	

Celda 10 (código)

```
# ============================================================
# H1: VISUALIZACIÓN - Salario EUR vs PPS lado a lado
# ============================================================
# plotly.express (px) es la versión "fácil" de Plotly.
# px.bar() crea un gráfico de barras interactivo automáticamente.
#
# El truco aquí es que Plotly necesita los datos en formato "long"
# para dibujar barras agrupadas. Es decir, en vez de tener:
#   geo | salario_min_eur | salario_min_pps
#   ES  | 1323            | 1400
#
# Necesita:
#   geo | tipo    | valor
#   ES  | EUR     | 1323
#   ES  | PPS     | 1400
#
# pd.melt() hace exactamente esa transformación (la misma que
# usé en el Notebook 1 para los datasets de Eurostat).
# ============================================================

# Preparo los datos en formato long para Plotly
df_barras = df_ranking[['geo', 'salario_min_eur', 'salario_min_pps']].melt(
    id_vars='geo',
    value_vars=['salario_min_eur', 'salario_min_pps'],
    var_name='tipo',
    value_name='salario'
)

# Renombro los valores para que se lean bien en la leyenda
df_barras['tipo'] = df_barras['tipo'].replace({
```

```
    'salario_min_eur': 'EUR (nominal)',
    'salario_min_pps': 'PPS (poder adquisitivo)'
})

fig = px.bar(
    df_barras,
    x='salario',               # eje horizontal: el valor del salario
    y='geo',                    # eje vertical: el país
    color='tipo',               # una barra de cada color por tipo
    orientation='h',            # horizontal (los países se leen mejor)
    barmode='group',            # las barras lado a lado, no apiladas
    title=f'H1: Salario mínimo nominal (EUR) vs real (PPS) - {anio_ref}',
    labels={'salario': 'Salario mínimo mensual', 'geo': 'País', 'tipo': 'Medida'},
    color_discrete_sequence=['#2196F3', '#FF5722']
)

# Ordeno los países por salario EUR de mayor a menor
orden_paises = df_ranking.sort_values('salario_min_eur', ascending=True)['geo'].tolist()
fig.update_yaxes(categoryorder='array', categoryarray=ord
```

Resultado:
`[GRÁFICO GENERADO]`

Celda 11 (código)

```
# ============================================================
# H1: ¿QUIÉN SUBE Y QUIÉN BAJA AL AJUSTAR POR PPS?
# ============================================================
# Barras horizontales con el cambio de posición de cada país.
# Verde a la derecha = sube (vive mejor de lo que parece).
# Rojo a la izquierda = baja (vive peor de lo que parece).
#
# Plotly me permite crear colores condicionales con np.where():
# np.where(condición, valor_si_true, valor_si_false)
# ============================================================

# Ordeno por cambio de posición
datos_cambio = df_ranking.sort_values('cambio_posicion', ascending=True).copy()

# Creo una columna de texto para el tooltip (lo que aparece al pasar el ratón)
datos_cambio['texto'] = datos_cambio.apply(
    lambda fila: f"EUR: {fila['salario_min_eur']:.0f}€ | PPS: {fila['salario_min_pps']:.0f} | "
                 f"Rank EUR: {fila['rank_eur']} → Rank PPS: {fila['rank_pps']}",
    axis=1
)

# Color condicional: verde si sube, rojo si baja
datos_cambio['color'] = np.where(datos_cambio['cambio_posicion'] > 0, 'Sube (vive mejor)',
                          np.where(datos_cambio['cambio_posicion'] < 0, 'Baja (vive peor)', 'Sin cambio'))

fig = px.bar(
    datos_cambio,
    x='cambio_posicion',
    y='geo',
    color='color',
    orientation='h',
    title=f'H1: Cambio en el ranking al ajustar por poder adquisitivo ({anio_ref})',
    labels={'cambio_posicion': 'Cambio de posiciones (EUR → PPS)', 'geo': 'País'},
    hover_data={'texto': True, 'cambio_posicion': True, 'color': False},
    color_discrete_map={
        'Sube (vive mejor)': '#4CAF50',
        'Baja (vive peor)': '#F44336',
        'Sin cambio': '#9E9E9E'
    }
)

fig.update_layout(height=700)
fig.show()

# Resumen numérico
print("Top 5 países que MÁS SUBEN al ajustar por PPS:")
top_suben = df_ranking.nlargest(5, 'cambio_posicion')[['geo', 'salario_min_eur', 'salario_min_pps', 'cambio_posicion'
print(top_suben.to_string(index=False))
```

```
print("\nT
```

Resultado:

```
[GRÁFICO GENERADO]
Top 5 países que MÁS SUBEN al ajustar por PPS:
geo  salario_min_eur  salario_min_pps  cambio_posicion
RO            703.0           1105.0                8
MK            513.0            936.0                7
TR            590.5           1062.0                6
LT            924.0           1132.0                4
HR            840.0           1102.0                3

Top 5 países que MÁS BAJAN al ajustar por PPS:
geo  salario_min_eur  salario_min_pps  cambio_posicion
EE            820.0
```

Celda 12 (código)

```
# ============================================================
# H1: SCATTER PLOT - ¿Cómo se relacionan EUR y PPS?
# ============================================================
# Cada punto es un país. Eje X = salario EUR, eje Y = salario PPS.
#
# Si EUR y PPS fueran idénticos, todos los puntos estarían en la
# diagonal (la línea roja punteada que añado como referencia).
# Los puntos por ENCIMA de la diagonal: PPS > EUR (más poder
# adquisitivo del que sugiere el salario nominal).
# Los puntos por DEBAJO: PPS < EUR (menos poder adquisitivo).
#
# px.scatter() crea el gráfico de dispersión.
# text='geo' pone el nombre del país al lado de cada punto.
# ============================================================

fig = px.scatter(
    df_ranking,
    x='salario_min_eur',
    y='salario_min_pps',
    text='geo',                 # Etiqueta con el código de país
    color='es_eu27',            # Color diferente para EU27 vs extra
    title=f'H1: Salario nominal vs poder adquisitivo real ({anio_ref})',
    labels={
        'salario_min_eur': 'Salario mínimo EUR (nominal)',
        'salario_min_pps': 'Salario mínimo PPS (poder adquisitivo)',
        'es_eu27': '¿EU27?'
    },
    hover_data={
        'geo': True,
        'salario_min_eur': ':.0f',
        'salario_min_pps': ':.0f',
        'cambio_posicion': True
    },
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

# Muevo las etiquetas un poco arriba para que no tapen el punto
fig.update_traces(textposition='top center', textfont_size=9)

# Línea diagonal de referencia: PPS = EUR
limite = max(df_ranking['salario_min_eur'].max(), df_ranking['salario_min_pps'].max()) + 100
fig.add_shape(
    type='line',
    x0=0, y0=0, x1=limite, y1=limite,
    line=dict(color='red', dash='dash', width=1)
)

fig.update_layout(height=650, width=800)
fig.show()
```

Resultado:

```
[GRÁFICO GENERADO]
```

Celda 13 (código)

```
# ============================================================
# H1: VALIDACIÓN ESTADÍSTICA
# ============================================================
# Aquí aplico el test de Pearson para medir la correlación entre
# el salario nominal (EUR) y el ajustado (PPS).
#
# ¿Qué espero encontrar?
# Una correlación alta y positiva (r cercano a 1): lógico, porque
# ambas son medidas de salario. PERO si r no es 1.0 perfecto,
# significa que el ajuste PPS SÍ cambia las cosas, confirmando H1.
#
# También calculo estadísticas descriptivas del cambio de posición
# para cuantificar cuánto "engaña" el salario nominal.
# ============================================================

print("=" * 60)
print("H1: VALIDACIÓN ESTADÍSTICA")
print("=" * 60)
print()

# Test de Pearson entre EUR y PPS
test_pearson(df_ranking, 'salario_min_eur', 'salario_min_pps',
             'Correlación EUR vs PPS')

# Estadísticas del cambio de posición
cambios = df_ranking['cambio_posicion']
print(f"Estadísticas del cambio de posición en el ranking:")
print(f"  Media de cambio: {cambios.mean():.1f} posiciones")
print(f"  Cambio máximo hacia arriba: +{cambios.max()} posiciones")
print(f"  Cambio máximo hacia abajo: {cambios.min()} posiciones")
print(f"  Países que cambian de posición: {(cambios != 0).sum()} de {len(cambios)}")
print(f"  Países que NO cambian: {(cambios == 0).sum()} de {len(cambios)}")
```

Resultado:

```
 ============================================================
H1: VALIDACIÓN ESTADÍSTICA
============================================================

  Correlación EUR vs PPS
    Observaciones válidas: 27
    Correlación (r): 0.9450 (fuerte, positiva)
    P-value: 0.000000
    ¿Significativa (p < 0.05)?: SÍ

Estadísticas del cambio de posición en el ranking:
  Media de cambio: 0.0 posiciones
  Cambio máximo hacia arriba: +8 posiciones
  Cambio máximo hacia abajo: -8 posiciones
  Países que cambi
```

Celda 14 (texto explicativo del notebook)

Conclusión H1: El salario nominal engaña **La hipótesis se valida.** El test de Pearson muestra una correlación fuerte y positiva (r = 0.945, p < 0.001) entre el salario en euros y el ajustado por poder adquisitivo. Esto es lógico: ambas miden salario. Pero la correlación NO es perfecta (no es 1.0), lo que confirma que el ajuste PPS sí cambia las cosas. La prueba más contundente está en los rankings: **21 de los 27 países cambian de posición** al ajustar por poder adquisitivo. El cambio llega hasta 8 posiciones en ambas direcciones. Los casos más llamativos: - **Rumanía sube 8 posiciones** (de la 19 a la 11): su salario nominal de 703€ parece bajo, pero el coste de vida es tan barato que equivale a 1.105 PPS, más que Portugal o Grecia. Un trabajador rumano puede comprar más con su salario mínimo que un portugués. - **Estonia baja 8 posiciones** (de la 16 a la 24): cobra 820€ pero su poder adquisitivo real es exactamente 820 PPS, lo que la hunde en el ranking cuando otros paíse

Celda 16 (texto explicativo del notebook)

H2: El salario mínimo como herramienta feminista **Hipótesis:** Los países con salario mínimo más alto tienen una brecha de género salarial más pequeña. Es decir, el salario mínimo actúa como herramienta de igualdad porque establece un suelo que beneficia desproporcionadamente a las mujeres (que están sobrerrepresentadas en los empleos peor pagados). **¿Qué es la brecha de género salarial?** Es la diferencia porcentual entre lo que cobran los hombres y lo que cobran las mujeres. Si la brecha es del 15%, significa que las mujeres cobran de media un 15% menos que los hombres. Una brecha del 0% sería igualdad perfecta. **Columnas que uso:** `salario_min_pps` (uso PPS y no EUR porque, como demostré en H1, el nominal engaña) y `brecha_genero` **¿Cómo la valido?** Si hay una correlación negativa significativa (r negativo, $p < 0.05$) entre salario mínimo y brecha de género, se confirma: a mayor salario mínimo, menor brecha.

Celda 17 (código)

```
# ============================================================
# H2: PREPARAR LOS DATOS
# ============================================================
# Necesito filas donde tenga AMBOS datos: salario mínimo Y brecha
# de género. Voy a trabajar con el año más reciente que tenga
# buena cobertura, igual que en H1.
#
# Uso salario_min_pps (no EUR) porque en H1 demostré que el
# nominal engaña. Sería incoherente usar EUR después de eso.
# ============================================================

df_h2 = df[['geo', 'anio', 'salario_min_pps', 'brecha_genero', 'es_eu27']].dropna()

print(f"Filas con salario PPS y brecha de género: {len(df_h2)}")
print(f"\nDatos por año:")
print(df_h2.groupby('anio')['geo'].nunique().to_string())
```

Resultado:

```
Filas con salario PPS y brecha de género: 344

Datos por año:
anio
2002    13
2006    21
2007    19
2008    18
2009    18
2010    19
2011    19
2012    19
2013    20
2014    25
2015    17
2016    18
2017    17
2018    23
2019    19
2020    19
2021    19
2022    19
2023     1
2024     1
```

Celda 18 (código)

```
# ============================================================
# H2: SCATTER PLOT - ¿Mayor salario = menor brecha?
# ============================================================
# Cada punto es un país en un año concreto.
# Eje X = salario mínimo en PPS
```

```
# Eje Y = brecha de género (%)
#
# Si H2 es correcta, los puntos deberían ir "hacia abajo":
# cuanto más a la derecha (más salario), más abajo (menos brecha).
#
# Uso TODOS los años disponibles (no solo 2024) para tener más
# observaciones y que el test de Pearson sea más robusto.
# Con un solo año tendría ~20 puntos; con todos tengo muchos más.
#
# px.scatter() con trendline='ols' añade una línea de tendencia.
# OLS = Ordinary Least Squares, es el método más básico para
# dibujar la "mejor recta" que pasa entre los puntos.
# Si la recta baja de izquierda a derecha = correlación negativa.
# ============================================================

fig = px.scatter(
    df_h2,
    x='salario_min_pps',
    y='brecha_genero',
    color='es_eu27',
    hover_data={'geo': True, 'anio': True, 'salario_min_pps': ':.0f', 'brecha_genero': ':.1f'},
    title='H2: ¿Mayor salario mínimo = menor brecha de género?',
    labels={
        'salario_min_pps': 'Salario mínimo (PPS)',
        'brecha_genero': 'Brecha de género (%)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_layout(height=600)
fig.show()
```

Resultado:
`[GRÁFICO GENERADO]`

Celda 19 (código)

```
# ============================================================
# H2: EVOLUCIÓN TEMPORAL
# ============================================================
# Además del scatter, quiero ver si la brecha de género ha ido
# bajando con el tiempo en los países que más han subido el
# salario mínimo.
#
# Selecciono algunos países representativos para que el gráfico
# sea legible (si pongo 38 líneas no se ve nada).
# ============================================================

# Países representativos: mezcla de salarios altos y bajos
paises_ejemplo = ['ES', 'DE', 'FR', 'PL', 'RO', 'EE', 'IT', 'PT', 'BG', 'SE']

df_evol = df[df['geo'].isin(paises_ejemplo)][['geo', 'anio', 'brecha_genero']].dropna()

fig = px.line(
    df_evol,
    x='anio',
    y='brecha_genero',
    color='geo',
    title='H2: Evolución de la brecha de género salarial en países seleccionados',
    labels={
        'anio': 'Año',
        'brecha_genero': 'Brecha de género (%)',
        'geo': 'País'
    },
    markers=True    # Añade un punto en cada dato (más fácil ver dónde hay datos)
)

fig.update_layout(height=500)
fig.show()
```

Resultado:
`[GRÁFICO GENERADO]`

Celda 20 (código)

```
# ============================================================
# H2: SNAPSHOT DEL ÚLTIMO AÑO
# ============================================================
# El scatter con todos los años es bueno para el test estadístico
# pero para la presentación es más claro un gráfico con UN solo
# año donde cada punto sea un país único.
#
# Busco el último año donde tenga datos de ambas columnas.
# ============================================================

# ¿Cuál es el último año con buena cobertura?
ultimo_anio_h2 = df_h2.groupby('anio')['geo'].nunique().idxmax()
print(f"Año con más países: {ultimo_anio_h2}")

df_h2_snap = df_h2[df_h2['anio'] == ultimo_anio_h2].copy()

fig = px.scatter(
    df_h2_snap,
    x='salario_min_pps',
    y='brecha_genero',
    text='geo',
    color='es_eu27',
    hover_data={'salario_min_pps': ':.0f', 'brecha_genero': ':.1f'},
    title=f'H2: Salario mínimo vs brecha de género ({ultimo_anio_h2})',
    labels={
        'salario_min_pps': 'Salario mínimo (PPS)',
        'brecha_genero': 'Brecha de género (%)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_traces(textposition='top center', textfont_size=9)
fig.update_layout(height=600)
fig.show()
```

Resultado:
```
Año con más países: 2014

[GRÁFICO GENERADO]
```

Celda 21 (código)

```
# ============================================================
# H2: VALIDACIÓN ESTADÍSTICA
# ============================================================
# Hago el test de Pearson dos veces:
# 1) Con TODOS los años (más observaciones = test más robusto)
# 2) Solo con el último año (foto actual, sin efecto temporal)
#
# ¿Por qué dos tests?
# Porque con todos los años puedo estar mezclando tendencias
# temporales (la brecha baja con el tiempo Y el salario sube con
# el tiempo, así que parecen correlacionados aunque la causa sea
# simplemente el paso del tiempo). El test del último año elimina
# esa confusión.
# ============================================================

print("=" * 60)
print("H2: VALIDACIÓN ESTADÍSTICA")
print("=" * 60)
```

```
print()

# Test 1: todos los años
print("Test 1: Todos los años disponibles")
test_pearson(df_h2, 'salario_min_pps', 'brecha_genero',
             'Salario PPS vs Brecha de género (todos los años)')

# Test 2: solo último año
print(f"Test 2: Solo año {ultimo_anio_h2}")
test_pearson(df_h2_snap, 'salario_min_pps', 'brecha_genero',
             f'Salario PPS vs Brecha de género ({ultimo_anio_h2})')
```

Resultado:

```
============================================================
H2: VALIDACIÓN ESTADÍSTICA
============================================================

Test 1: Todos los años disponibles
  Salario PPS vs Brecha de género (todos los años)
    Observaciones válidas: 344
    Correlación (r): -0.2990 (débil, negativa)
    P-value: 0.000000
    ¿Significativa (p < 0.05)?: SÍ

Test 2: Solo año 2014
  Salario PPS vs Brecha de género (2014)
    Observaciones válidas: 25
    Correlación (r): -0.1158 (débil
(np.float64(-0.11576998617933158), np.float64(0.5815832430966691))
```

Celda 22 (texto explicativo del notebook)

Conclusión H2: El salario mínimo como herramienta feminista **La hipótesis se valida parcialmente.** Los resultados muestran una imagen con matices: **Test con todos los años (344 observaciones):** la correlación es negativa (r = -0.299) y estadísticamente significativa (p < 0.001). Esto significa que, mirando el conjunto histórico, SÍ existe una tendencia: los países con salario mínimo más alto tienden a tener una brecha de género más pequeña. Sin embargo, la correlación es débil (r < 0.3), lo que indica que el salario mínimo explica solo una pequeña parte de la brecha. **Test con un solo año, 2014 (25 observaciones):** la correlación sigue siendo negativa (r = -0.116) pero NO es significativa (p = 0.58). Con un 58% de probabilidad de que sea casualidad, no podemos afirmar que la relación exista en una foto fija de un año concreto. **¿Por qué esta diferencia?** Con todos los años mezclamos dos efectos temporales: el salario mínimo sube con el tiempo Y la brecha de género baja

Celda 24 (texto explicativo del notebook)

H3: El salario mínimo no da para vivir **Hipótesis:** El salario mínimo es insuficiente para cubrir el coste de la vivienda en la mayoría de países europeos. A pesar de que los salarios suben nominalmente, los precios de la vivienda suben más rápido, haciendo que la emancipación sea cada vez más difícil. **¿Qué es el índice de precios de vivienda?** Es un número que mide cuánto han subido (o bajado) los precios de la vivienda respecto a un año base (2010 = 100). Si el índice es 150, significa que los precios han subido un 50% desde 2010. No me dice el precio absoluto de un piso, sino la tendencia. **Columnas que uso:** `salario_min_pps` y `indice_vivienda` **¿Cómo la valido?** Si la vivienda sube más rápido que el salario (correlación positiva entre índice de vivienda y salario, pero con la vivienda creciendo proporcionalmente más), la hipótesis se confirma. También comparo las tasas de crecimiento de ambas variables a lo largo del tiempo.

Celda 25 (código)

```
# ============================================================
### H3: PREPARAR LOS DATOS
# ============================================================
# Necesito filas donde tenga salario mínimo Y índice de vivienda.
# El índice de vivienda empieza en 2015, así que los años
# anteriores no me sirven para esta hipótesis.
# ============================================================

df_h3 = df[['geo', 'anio', 'salario_min_pps', 'indice_vivienda', 'es_eu27']].dropna()
```

```
print(f"Filas con salario PPS e índice de vivienda: {len(df_h3)}")
print(f"\nDatos por año:")
print(df_h3.groupby('anio')['geo'].nunique().to_string())
print(f"\nRango temporal: {df_h3['anio'].min()} - {df_h3['anio'].max()}")
```

Resultado:

```
Filas con salario PPS e índice de vivienda: 203

Datos por año:
anio
2015    20
2016    21
2017    21
2018    21
2019    21
2020    20
2021    20
2022    19
2023    21
2024    19

Rango temporal: 2015 - 2024
```

Celda 27 (código)

```
# ============================================================
# H3: EVOLUCIÓN COMPARADA - ¿Qué sube más rápido?
# ============================================================
# La pregunta clave de H3 no es si el salario y la vivienda están
# correlacionados, sino CUÁL CRECE MÁS RÁPIDO.
#
# Para comparar dos cosas que se miden en unidades diferentes
# (euros vs índice), las normalizo: convierto ambas a "porcentaje
# de cambio desde el primer año". Así las puedo poner en el mismo
# gráfico.
#
# Selecciono algunos países representativos para que sea legible.
# ============================================================

paises_h3 = ['ES', 'DE', 'FR', 'PL', 'RO', 'IE', 'PT', 'NL', 'CZ', 'BG']

# Filtro solo los países seleccionados y ordeno por año
df_evol_h3 = df_h3[df_h3['geo'].isin(paises_h3)].sort_values(['geo', 'anio']).copy()

# Para cada país, calculo el crecimiento porcentual desde su primer año
# Es decir: (valor_actual / valor_primer_año) * 100
# Así el primer año de cada país empieza en 100 y veo cuánto sube cada uno

resultados = []
for pais in paises_h3:
    datos_pais = df_evol_h3[df_evol_h3['geo'] == pais].copy()
    if len(datos_pais) < 2:
        continue

    # Valor del primer año disponible como referencia (base = 100)
    base_salario = datos_pais['salario_min_pps'].iloc[0]
    base_vivienda = datos_pais['indice_vivienda'].iloc[0]

    datos_pais['salario_idx'] = (datos_pais['salario_min_pps'] / base_salario) * 100
    datos_pais['vivienda_idx'] = (datos_pais['indice_vivienda'] / base_vivienda) * 100

    resultados.append(datos_pais)

df_idx = pd.concat(resultados)

print(f"Países analizados: {df_idx['geo'].nunique()}")
print(f"Años: {df_idx['anio'].min()} - {df_idx['anio'].max()}")
df_idx[['geo', 'anio', 'salario_min_pps', 'salario_idx', 'indice_vivienda', 'vivienda_idx']].head(10)
```

Resultado:

```
Países analizados: 10
Años: 2015 - 2024

    geo  anio  salario_min_pps  salario_idx  indice_vivienda  vivienda_idx
328  ES  2015            798.0   100.000000            74.32    100.000000
329  ES  2016            802.0   100.501253            77.76    104.628633
330  ES  2017            872.0   109.273183            82.57    111.100646
```

Celda 28 (código)

```
# ============================================================
# H3: GRÁFICO - ¿La vivienda sube más rápido que el salario?
# ============================================================
# Para cada país dibujo dos líneas: salario (azul) y vivienda (rojo).
# Si la línea roja sube más que la azul, la vivienda crece más
# rápido que el salario y H3 se confirma para ese país.
#
# Uso px.line con facet_col para crear un mini-gráfico por país.
# facet_col_wrap=5 pone 5 gráficos por fila.
# ============================================================

# Primero paso a formato long para que Plotly pueda colorear por tipo
df_idx_long = df_idx.melt(
    id_vars=['geo', 'anio'],
    value_vars=['salario_idx', 'vivienda_idx'],
    var_name='indicador',
    value_name='indice_base100'
)

# Renombro para que la leyenda sea legible
df_idx_long['indicador'] = df_idx_long['indicador'].replace({
    'salario_idx': 'Salario mínimo (PPS)',
    'vivienda_idx': 'Precio vivienda'
})

fig = px.line(
    df_idx_long,
    x='anio',
    y='indice_base100',
    color='indicador',
    facet_col='geo',            # Un mini-gráfico por país
    facet_col_wrap=5,           # 5 por fila
    title='H3: Crecimiento del salario mínimo vs precio de vivienda (base 100)',
    labels={'anio': 'Año', 'indice_base100': 'Índice (base=100)', 'indicador': ''},
    color_discrete_sequence=['#2196F3', '#F44336'],
    markers=True
)

fig.update_layout(height=600)
fig.show()
```

Resultado:

```
[GRÁFICO GENERADO]
```

Celda 29 (código)

```
# ============================================================
# H3: SCATTER PLOT - Relación entre salario y vivienda
# ============================================================
# Cada punto es un país en un año.
# Si la vivienda y el salario suben juntos, los puntos irán
# hacia arriba y a la derecha. Pero lo importante es la proporción:
# ¿sube más la vivienda que el salario?
# ============================================================

fig = px.scatter(
```

```
    df_h3,
    x='salario_min_pps',
    y='indice_vivienda',
    color='es_eu27',
    hover_data={'geo': True, 'anio': True, 'salario_min_pps': ':.0f', 'indice_vivienda': ':.1f'},
    title='H3: Salario mínimo vs índice de precios de vivienda',
    labels={
        'salario_min_pps': 'Salario mínimo (PPS)',
        'indice_vivienda': 'Índice de precios de vivienda (2010=100)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_layout(height=600)
fig.show()
```

Resultado:

`[GRÁFICO GENERADO]`

Celda 30 (código)

```
# ============================================================
# H3: VALIDACIÓN ESTADÍSTICA
# ============================================================
# Test de Pearson entre salario y vivienda.
# Además, calculo para cuántos países la vivienda ha crecido
# más rápido que el salario (la pregunta central de H3).
# ============================================================

print("=" * 60)
print("H3: VALIDACIÓN ESTADÍSTICA")
print("=" * 60)
print()

# Test de Pearson
test_pearson(df_h3, 'salario_min_pps', 'indice_vivienda',
             'Salario PPS vs Índice de vivienda (todos los años)')

# ¿En cuántos países la vivienda crece más rápido que el salario?
print("Comparación de crecimiento por país:")
print("-" * 50)

conteo_vivienda_gana = 0
conteo_salario_gana = 0

for pais in df_idx['geo'].unique():
    datos_pais = df_idx[df_idx['geo'] == pais]
    crec_salario = datos_pais['salario_idx'].iloc[-1] - 100  # crecimiento desde base
    crec_vivienda = datos_pais['vivienda_idx'].iloc[-1] - 100

    ganador = "VIVIENDA" if crec_vivienda > crec_salario else "SALARIO"
    if ganador == "VIVIENDA":
        conteo_vivienda_gana += 1
    else:
        conteo_salario_gana += 1

    print(f"  {pais}: salario +{crec_salario:.1f}% | vivienda +{crec_vivienda:.1f}% → gana {ganador}")

print(f"\nResumen: en {conteo_vivienda_gana} de {conteo_vivienda_gana + conteo_salario_gana} países "
      f"la vivienda crece más rápido que el salario")
```

Resultado:

```
 ============================================================
H3: VALIDACIÓN ESTADÍSTICA
============================================================

 Salario PPS vs Índice de vivienda (todos los años)
   Observaciones válidas: 203
   Correlación (r): 0.0444 (débil, positiva)
```

```
   P-value: 0.529164
   ¿Significativa (p < 0.05)?: NO

Comparación de crecimiento por país:
--------------------------------------------------
  ES: salario +82.5% | vivienda +60.3% → gana SALARIO
  DE: salario +40.1%
```

Celda 31 (texto explicativo del notebook)

Conclusión H3: El salario mínimo no da para vivir **La hipótesis se valida por la vía del crecimiento, aunque no por correlación directa.** Esto es importante de entender: el test de Pearson NO da significativo (r = 0.044, p = 0.53). Esto significa que no hay una relación lineal entre el nivel de salario de un país y su índice de vivienda. Un país con salario alto no tiene necesariamente vivienda más cara medida en índice, y viceversa. Pero esa NO era realmente la pregunta. La pregunta de H3 es: ¿la vivienda sube más rápido que el salario? Y ahí los datos son contundentes: **En 8 de los 10 países analizados, los precios de la vivienda crecen más rápido que el salario mínimo.** Los casos más extremos: - **Portugal:** el salario subió un 61.5% pero la vivienda subió un 124.4%. La vivienda crece el doble de rápido que el salario. - **Países Bajos:** salario +37.5% vs vivienda +98.2%. Casi el triple. - **Chequia:** salario +68.5% vs vivienda +122.2%. Casi el doble. - **Irlanda:**

Celda 33 (texto explicativo del notebook)

H4: Trabajar no te saca de pobre (working poor) **Hipótesis:** Tener empleo no garantiza salir de la pobreza. Existe un porcentaje significativo de "trabajadores pobres" (working poor) en Europa, y un salario mínimo bajo está asociado a mayor pobreza laboral. **¿Qué es la pobreza laboral?** Es el porcentaje de personas que TRABAJAN pero aun así están por debajo del umbral de pobreza de su país. En Eurostat, el indicador `ilc_iw01` mide esto para la población de 18 a 64 años con empleo. Si la tasa es del 10%, significa que 1 de cada 10 trabajadores es pobre a pesar de tener trabajo. **¿Qué es la tasa de desempleo?** El porcentaje de personas en edad de trabajar que buscan empleo activamente pero no lo encuentran. La uso como contexto: ¿los países con más desempleo también tienen más pobreza laboral, o son problemas independientes? **Columnas que uso:** `salario_min_pps`, `pobreza_laboral`, `tasa_desempleo` **¿Cómo la valido?** Si hay correlación negativa significativa entre sala

Celda 34 (código)

```
# ============================================================
# H4: PREPARAR LOS DATOS
# ============================================================
# Para esta hipótesis necesito tres columnas: salario, pobreza
# laboral y desempleo. Miro la disponibilidad de cada combinación.
# ============================================================

# Relación principal: salario vs pobreza laboral
df_h4 = df[['geo', 'anio', 'salario_min_pps', 'pobreza_laboral', 'tasa_desempleo', 'es_eu27']].dropna(
    subset=['salario_min_pps', 'pobreza_laboral']  # Exijo estas dos, desempleo puede faltar
)

print(f"Filas con salario PPS y pobreza laboral: {len(df_h4)}")
print(f"\nDatos por año:")
print(df_h4.groupby('anio')['geo'].nunique().to_string())
print(f"\nRango temporal: {df_h4['anio'].min()} - {df_h4['anio'].max()}")
print(f"Países: {df_h4['geo'].nunique()}")
```

Resultado:

```
Filas con salario PPS y pobreza laboral: 505

Datos por año:
anio
2003     4
2004     8
2005    18
2006    20
2007    21
2008    21
2009    21
```

```
2010    22
2011    22
2012    22
2013    25
2014    25
2015    26
2016    26
2017    27
2018    27
2019    26
2020    26
2021    26
2022    26
2023    26
2024    24
2025    16

Rango temporal: 2003 - 2025
Países: 28
```

Celda 35 (código)

```
# ============================================================
# H4: SCATTER PLOT - ¿Mayor salario = menos trabajadores pobres?
# ============================================================
# Si H4 es correcta, la nube de puntos debería ir "hacia abajo":
# más salario (derecha) = menos pobreza laboral (abajo).
# ============================================================

fig = px.scatter(
    df_h4,
    x='salario_min_pps',
    y='pobreza_laboral',
    color='es_eu27',
    hover_data={'geo': True, 'anio': True, 'salario_min_pps': ':.0f', 'pobreza_laboral': ':.1f'},
    title='H4: ¿Mayor salario mínimo = menos trabajadores pobres?',
    labels={
        'salario_min_pps': 'Salario mínimo (PPS)',
        'pobreza_laboral': 'Tasa de pobreza laboral (%)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_layout(height=600)
fig.show()
```

Resultado:

```
[GRÁFICO GENERADO]
```

Celda 36 (código)

```
# ============================================================
# H4: SNAPSHOT - Un año concreto para ver cada país
# ============================================================

ultimo_anio_h4 = df_h4.groupby('anio')['geo'].nunique().idxmax()
print(f"Año con más países: {ultimo_anio_h4}")

df_h4_snap = df_h4[df_h4['anio'] == ultimo_anio_h4].copy()

fig = px.scatter(
    df_h4_snap,
    x='salario_min_pps',
    y='pobreza_laboral',
```

```
    text='geo',
    color='es_eu27',
    hover_data={'salario_min_pps': ':.0f', 'pobreza_laboral': ':.1f'},
    title=f'H4: Salario mínimo vs pobreza laboral ({ultimo_anio_h4})',
    labels={
        'salario_min_pps': 'Salario mínimo (PPS)',
        'pobreza_laboral': 'Tasa de pobreza laboral (%)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_traces(textposition='top center', textfont_size=9)
fig.update_layout(height=600)
fig.show()
```

Resultado:

```
Año con más países: 2017

[GRÁFICO GENERADO]
```

Celda 37 (código)

```
# ============================================================
# H4: ¿POBREZA LABORAL Y DESEMPLEO SON LO MISMO?
# ============================================================
# Pregunta interesante: ¿los países con más desempleo tienen más
# working poor, o son problemas distintos?
#
# Si NO están correlacionados, eso refuerza el argumento de H4:
# la pobreza laboral no se resuelve simplemente "creando empleo",
# porque el problema es la CALIDAD del empleo (salarios bajos).
# ============================================================

df_h4_desemp = df_h4.dropna(subset=['tasa_desempleo'])

fig = px.scatter(
    df_h4_desemp,
    x='tasa_desempleo',
    y='pobreza_laboral',
    color='es_eu27',
    hover_data={'geo': True, 'anio': True},
    title='H4: ¿Más desempleo = más pobreza laboral? (no necesariamente)',
    labels={
        'tasa_desempleo': 'Tasa de desempleo (%)',
        'pobreza_laboral': 'Tasa de pobreza laboral (%)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_layout(height=600)
fig.show()
```

Resultado:

```
[GRÁFICO GENERADO]
```

Celda 38 (código)

```
# ============================================================
# H4: VALIDACIÓN ESTADÍSTICA
# ============================================================

print("=" * 60)
```

```
print("H4: VALIDACIÓN ESTADÍSTICA")
print("=" * 60)
print()

# Test principal: salario vs pobreza laboral (todos los años)
print("Test 1: Salario vs pobreza laboral (todos los años)")
test_pearson(df_h4, 'salario_min_pps', 'pobreza_laboral',
             'Salario PPS vs Pobreza laboral')

# Test del snapshot
print(f"Test 2: Salario vs pobreza laboral (solo {ultimo_anio_h4})")
test_pearson(df_h4_snap, 'salario_min_pps', 'pobreza_laboral',
             f'Salario PPS vs Pobreza laboral ({ultimo_anio_h4})')

# Test complementario: desempleo vs pobreza laboral
print("Test 3: Desempleo vs pobreza laboral (contexto)")
test_pearson(df_h4_desemp, 'tasa_desempleo', 'pobreza_laboral',
             'Desempleo vs Pobreza laboral')

# Estadísticas descriptivas de pobreza laboral
print("Estadísticas de pobreza laboral en Europa:")
print(f"  Media: {df_h4['pobreza_laboral'].mean():.1f}%")
print(f"  Mediana: {df_h4['pobreza_laboral'].median():.1f}%")
print(f"  Mínimo: {df_h4['pobreza_laboral'].min():.1f}%")
print(f"  Máximo: {df_h4['pobreza_laboral'].max():.1f}%")
```

Resultado:

```
 ============================================================
H4: VALIDACIÓN ESTADÍSTICA
============================================================

Test 1: Salario vs pobreza laboral (todos los años)
  Salario PPS vs Pobreza laboral
    Observaciones válidas: 505
    Correlación (r): -0.2451 (débil, negativa)
    P-value: 0.000000
    ¿Significativa (p < 0.05)?: SÍ

Test 2: Salario vs pobreza laboral (solo 2017)
  Salario PPS vs Pobreza laboral (2017)
    Observaciones válidas: 27
    Correlac
```

Celda 39 (texto explicativo del notebook)

Conclusión H4: Trabajar no te saca de pobre **La hipótesis se valida.** La media de pobreza laboral en Europa es del 8.6%. Eso significa que, de media, casi 1 de cada 10 trabajadores europeos es pobre a pesar de tener empleo. El máximo llega al 19.7%: en algún país europeo, 1 de cada 5 trabajadores no llega a fin de mes aunque trabaje. **Test principal (505 observaciones):** la correlación entre salario mínimo y pobreza laboral es negativa (r = -0.245) y significativa (p < 0.001). A mayor salario mínimo, menos trabajadores pobres. La relación es débil pero real: el salario mínimo ayuda, aunque no es el único factor. **Test de un solo año, 2017 (27 países):** la correlación es más fuerte (r = -0.323, moderada) pero no alcanza la significancia estadística (p = 0.10). Esto se debe al tamaño de muestra: con solo 27 observaciones, Pearson necesita una correlación más intensa para descartar el azar. Sin embargo, la dirección (negativa) es consistente con el test general. **El hallaz

Celda 41 (texto explicativo del notebook)

H5: La Generación Z trabaja más para vivir peor **Hipótesis:** A pesar de que el salario mínimo sube cada año en términos nominales, cuando se ajusta por la inflación (salario real), el poder adquisitivo de los trabajadores jóvenes se ha estancado o incluso reducido. La Generación Z (nacidos ~1997-2012) se incorpora a un mercado laboral donde los salarios reales compran menos que lo que compraban los salarios de sus padres. **¿Qué es la inflación (HICP)?** El Índice Armonizado de Precios al Consumo mide cuánto suben los precios de los bienes y servicios básicos (comida, transporte, energía...). Si el HICP es 120 y el año base era 100, los precios han subido un 20%. El indicador de Eurostat (`prc_hicp_aind`) usa 2015 como año base (2015 = 100). **¿Qué es el salario real?** Es el salario ajustado por inflación. Si mi salario sube un 10% pero los precios suben un 15%, mi salario real ha BAJADO un 5%. La fórmula es: `salario_real = (salario_nominal / HICP) * 100` **Columnas que uso:

Celda 42 (código)

```
# ============================================================
# H5: PREPARAR DATOS Y CALCULAR SALARIO REAL
# ============================================================
# El salario real se calcula así:
#   salario_real = (salario_nominal / HICP) * 100
#
# ¿Por qué?
# Si gano 1000€ y el HICP es 120 (los precios han subido un 20%
# desde 2015), mi salario real es (1000 / 120) * 100 = 833€.
# Eso significa que mis 1000€ de hoy compran lo mismo que 833€
# compraban en 2015.
#
# Multiplico por 100 para que el resultado esté en "euros de 2015",
# que es una forma estándar de expresar salarios reales.
# ============================================================

df_h5 = df[['geo', 'anio', 'salario_min_eur', 'salario_min_pps', 'inflacion_hicp', 'es_eu27']].dropna(
    subset=['salario_min_eur', 'inflacion_hicp']
)

# Calculo el salario real (en euros constantes de 2015)
df_h5['salario_real'] = (df_h5['salario_min_eur'] / df_h5['inflacion_hicp']) * 100

print(f"Filas con salario e inflación: {len(df_h5)}")
print(f"Rango temporal: {df_h5['anio'].min()} - {df_h5['anio'].max()}")
print(f"Países: {df_h5['geo'].nunique()}")
print(f"\nEjemplo con España:")
print(df_h5[df_h5['geo'] == 'ES'][['anio', 'salario_min_eur', 'inflacion_hicp', 'salario_real']].to_string(index=Fals
```

Resultado:

```
Filas con salario e inflación: 646
Rango temporal: 1999 - 2025
Países: 28

Ejemplo con España:
 anio  salario_min_eur  inflacion_hicp  salario_real
 1999           486.0           68.82    706.190061
 2000           496.0           71.22    696.433586
 2001           506.0           73.23    690.973645
 2002           516.0           75.86    680.200369
 2003           526.0           78.21    672.548267
 2004           555.0           80.60    688.585608
 2005           599.0           8
```

Celda 43 (código)

```
# ============================================================
# H5: EVOLUCIÓN - Salario nominal vs salario real
# ============================================================
# Para cada país dibujo dos líneas:
# - Azul: salario nominal (lo que dice la nómina)
# - Roja: salario real (lo que realmente puedo comprar)
#
# Si la línea azul sube pero la roja se queda plana o baja,
# H5 se confirma: el sueldo sube en el papel pero no en la vida.
# ============================================================

paises_h5 = ['ES', 'DE', 'FR', 'PL', 'RO', 'PT', 'EE', 'BG', 'IE', 'CZ']

df_h5_sel = df_h5[df_h5['geo'].isin(paises_h5)].copy()

# Paso a formato long para que Plotly pueda colorear por tipo
df_h5_long = df_h5_sel.melt(
    id_vars=['geo', 'anio'],
    value_vars=['salario_min_eur', 'salario_real'],
    var_name='tipo_salario',
    value_name='euros'
)
```

```
df_h5_long['tipo_salario'] = df_h5_long['tipo_salario'].replace({
    'salario_min_eur': 'Salario nominal (€)',
    'salario_real': 'Salario real (€ de 2015)'
})

fig = px.line(
    df_h5_long,
    x='anio',
    y='euros',
    color='tipo_salario',
    facet_col='geo',
    facet_col_wrap=5,
    title='H5: Salario mínimo nominal vs real (ajustado por inflación)',
    labels={'anio': 'Año', 'euros': '€', 'tipo_salario': ''},
    color_discrete_sequence=['#2196F3', '#F44336'],
    markers=True
)

fig.update_layout(height=600)
fig.show()
```

Resultado:
`[GRÁFICO GENERADO]`

Celda 44 (código)

```
# ============================================================
# H5: ¿EN CUÁNTOS PAÍSES HA BAJADO EL SALARIO REAL?
# ============================================================
# Para cada país comparo el salario real del primer año disponible
# con el del último. Si el último es menor, el trabajador puede
# comprar MENOS que antes a pesar de cobrar más euros.
# ============================================================

print("Evolución del salario real por país:")
print("=" * 65)

conteo_baja = 0
conteo_sube = 0
resultados_h5 = []

for pais in sorted(df_h5['geo'].unique()):
    datos_pais = df_h5[df_h5['geo'] == pais].sort_values('anio')
    if len(datos_pais) < 2:
        continue

    primer_anio = datos_pais['anio'].iloc[0]
    ultimo_anio = datos_pais['anio'].iloc[-1]
    salario_nom_inicio = datos_pais['salario_min_eur'].iloc[0]
    salario_nom_fin = datos_pais['salario_min_eur'].iloc[-1]
    salario_real_inicio = datos_pais['salario_real'].iloc[0]
    salario_real_fin = datos_pais['salario_real'].iloc[-1]

    cambio_nominal = ((salario_nom_fin / salario_nom_inicio) - 1) * 100
    cambio_real = ((salario_real_fin / salario_real_inicio) - 1) * 100

    if cambio_real < 0:
        conteo_baja += 1
        estado = "BAJA"
    else:
        conteo_sube += 1
        estado = "SUBE"

    resultados_h5.append({
        'geo': pais,
        'periodo': f"{primer_anio}-{ultimo_anio}",
        'nominal': f"+{cambio_nominal:.1f}%",
        'real': f"{cambio_real:+.1f}%",
        'estado': estado
    })

    print(f"  {pais} ({primer_anio}-{ultimo_anio}): "
          f"nominal +{cambio_nominal:.1f}% | real {cambio_real:+.1f}% → {estado}")
```

```
total = conteo_baja + conteo_sube
print(f"\nResumen:")
print(f"  Salario real SUBE: {conteo_sube} países ({conteo_sube/total*100:.0f}%)")
print(f"  Salario real BAJA: {conteo_baja} países ({conteo_baja/total*100:.0f}%)")
```

Resultado:

```
Evolución del salario real por país:
================================================================
  AL (2016-2025): nominal +155.0% | real +94.0% → SUBE
  BE (1999-2025): nominal +92.7% | real +3.8% → SUBE
  BG (1999-2025): nominal +1595.4% | real +483.2% → SUBE
  CY (2023-2025): nominal +6.4% | real +3.2% → SUBE
  CZ (1999-2025): nominal +772.8% | real +298.8% → SUBE
  DE (2015-2025): nominal +49.7% | real +13.5% → SUBE
  EE (1999-2025): nominal +1007.5% | real +281.6% → SUBE
  EL (1999-20
```

Celda 45 (código)

```
# ============================================================
# H5: ¿MÁS INFLACIÓN = MENOS SALARIO REAL?
# ============================================================
# Si H5 es correcta, los países con más inflación deberían tener
# un salario real que crece menos (o que baja).
# ============================================================

fig = px.scatter(
    df_h5,
    x='inflacion_hicp',
    y='salario_real',
    color='es_eu27',
    hover_data={'geo': True, 'anio': True, 'salario_min_eur': ':.0f', 'inflacion_hicp': ':.1f'},
    title='H5: Inflación (HICP) vs salario mínimo real',
    labels={
        'inflacion_hicp': 'Índice de inflación (HICP, 2015=100)',
        'salario_real': 'Salario mínimo real (€ de 2015)',
        'es_eu27': '¿EU27?'
    },
    trendline='ols',
    color_discrete_map={True: '#2196F3', False: '#FF9800'}
)

fig.update_layout(height=600)
fig.show()
```

Resultado:

```
[GRÁFICO GENERADO]
```

Celda 46 (código)

```
# ============================================================
# H5: VALIDACIÓN ESTADÍSTICA
# ============================================================

print("=" * 60)
print("H5: VALIDACIÓN ESTADÍSTICA")
print("=" * 60)
print()

# Test 1: inflación vs salario real
print("Test 1: Inflación vs salario real")
test_pearson(df_h5, 'inflacion_hicp', 'salario_real',
             'Inflación (HICP) vs Salario real')
```

```
# Test 2: inflación vs salario nominal (para comparar)
print("Test 2: Inflación vs salario nominal (comparación)")
test_pearson(df_h5, 'inflacion_hicp', 'salario_min_eur',
             'Inflación (HICP) vs Salario nominal')

# Estadísticas del salario real
print("Estadísticas del salario real (€ de 2015):")
print(f"  Media: {df_h5['salario_real'].mean():.0f}€")
print(f"  Mediana: {df_h5['salario_real'].median():.0f}€")
print(f"  Mínimo: {df_h5['salario_real'].min():.0f}€")
print(f"  Máximo: {df_h5['salario_real'].max():.0f}€")
```

Resultado:

```
============================================================
H5: VALIDACIÓN ESTADÍSTICA
============================================================

Test 1: Inflación vs salario real
  Inflación (HICP) vs Salario real
    Observaciones válidas: 646
    Correlación (r): -0.0595 (débil, negativa)
    P-value: 0.130714
    ¿Significativa (p < 0.05)?: NO

Test 2: Inflación vs salario nominal (comparación)
  Inflación (HICP) vs Salario nominal
    Observaciones válidas: 646
    Correlación (r): 0.10
```

Celda 47 (texto explicativo del notebook)

Conclusión H5: La Generación Z trabaja más para vivir peor **La hipótesis NO se valida con los datos disponibles.** Los resultados contradicen lo esperado: en 27 de 28 países analizados, el salario real (ajustado por inflación) ha SUBIDO a lo largo del tiempo. El único país donde ha bajado es Turquía (-97.9%), un caso extremo de hiperinflación que no es representativo del resto de Europa. **¿Qué dicen los tests?** - La correlación entre inflación y salario real no es significativa (r = -0.06, p = 0.13). No hay evidencia estadística de que más inflación destruya el poder adquisitivo del salario mínimo en Europa a largo plazo. - La correlación entre inflación y salario nominal sí es significativa (r = 0.10, p = 0.009), lo que indica que los gobiernos tienden a subir el salario nominal cuando la inflación sube. Es decir, hay un mecanismo de ajuste (aunque imperfecto). **Sin embargo, la historia tiene matices importantes:** El salario real sube, pero la velocidad importa. Miremos

Celda 49 (texto explicativo del notebook)

Resumen de resultados | Hipótesis | Pregunta | Resultado | Pearson (r) | P-value | Validada | |----------|---------|----------|----------|---------|---------| | H1 | ¿El salario nominal engaña? | 21 de 27 países cambian de posición al ajustar por PPS | 0.945 | < 0.001 | Sí | | H2 | ¿El salario mínimo reduce la brecha de género? | Correlación débil pero significativa a largo plazo | -0.299 | < 0.001 | Parcialmente | | H3 | ¿La vivienda sube más que el salario? | En 8 de 10 países la vivienda crece más rápido | 0.044 | 0.529 | Sí (por crecimiento, no por correlación) | | H4 | ¿Trabajar no te saca de pobre? | 8.6% de trabajadores europeos son pobres | -0.245 | < 0.001 | Sí | | H5 | ¿La Gen Z vive peor que sus padres? | El salario real sube, pero se estanca en Europa Occidental | -0.060 | 0.131 | No (pero con matices importantes) | ### Hallazgos principales 1. **Comparar salarios nominales entre países es engañoso.** El ajuste por poder adquisitivo cambia el ranking de 21 de 2

06. La sección del País Vasco

Esta sección NO es una hipótesis del notebook. Es un ejercicio de **contextualización**: tomo el dato político de 1.500€ y lo sitúo en el mapa europeo usando lo que demostré en H1 (el nominal engaña, hay que mirar PPS).

El cálculo

```
España 2024: 1.323€ = 1.456 PPS
Ratio PPS/EUR de España: 1.456 / 1.323 = 1.1005
País Vasco: 1.500€ × 1.1005 = 1.651 PPS
```

¿Por qué uso el ratio de España? Porque Eurostat no tiene datos a nivel de CCAA. El ratio de España es una aproximación. En realidad, el coste de vida en Euskadi es algo más alto que la media española, así que el PPS real del País Vasco sería algo menor. Pero para contextualizar a nivel europeo es razonable.

¿Dónde queda 1.651 PPS?

País/Región	Nominal (€)	PPS
Alemania	2.054	1.891
Países Bajos	2.102	1.812
Bélgica	2.032	1.743
País Vasco (propuesta)	**1.500**	**1.651**
Francia	1.767	1.589
España (actual)	1.323	1.456
Portugal	957	1.100

1.651 PPS supera a Francia (1.589) y se acerca a Bélgica (1.743). No es un outlier europeo: es una convergencia con la media alta.

> **El mensaje de la sección País Vasco:** 1.500€ en Euskadi es una propuesta ambiciosa pero coherente con los estándares europeos. Sin embargo, los datos del EDA también dicen que la cifra sola no basta: sin política de vivienda (H3), sin medidas de igualdad (H2), y sin abordar la precariedad contractual que genera pobreza laboral (H4), un SM de 1.500€ será necesario pero insuficiente para garantizar dignidad económica real.

07. La web: qué muestra cada sección

La web (cuantovaletutrabajo.lovable.app) tiene esta estructura:

Sección	Qué contiene
Portada	Título, cita gancho ("En 2006, una semana de salario pagaba un mes de alquiler. En 2025, no cubre ni la mitad."), scroll para explorar.
Presentación	Las 5 preguntas del proyecto. Contexto del debate de los 1.500€.
Metodología	6 cards explicando PPS, correlación, significancia, fuentes, limpieza. Accesible para no-científicos. Sección expandible de detalle técnico.
Mapa interactivo	38 países coloreados por SM en PPS. Click para ver detalle de cada país. UE en verde, no-UE en azul, sin SM legal en línea punteada.
H1 a H5	Cada una con: título grande, subtítulo gancho, texto explicativo, gráfico principal, análisis en profundidad (VALIDADA/PARCIAL/REFUTADA), datos clave en cards, casos destacados, cita-resumen.
País Vasco	Foco local. 4 stats cards (1.651 PPS, +13.4%, ~Francia, +10.1%). Gráfico de barras comparativas. Texto de contextualización.
Conclusiones	5 cards de resumen (una por hipótesis). 5 hallazgos principales. Implicaciones políticas. Botones de descarga (notebook, datos, notas metodológicas).

08. Chuleta rápida

Comandos de Pandas que usé

Qué quiero hacer	Comando	Explicación en cristiano
Ver primeras filas	`df.head()`	Muestra las primeras 5 filas para ver cómo son los datos
Dimensiones	`df.shape`	Dice cuántas filas y columnas tiene: (filas, columnas)
Tipos de datos	`df.dtypes`	Dice si cada columna es número, texto, fecha, etc.
Estadísticas	`df.describe()`	Media, mínimo, máximo, etc. de cada columna numérica
Contar nulos	`df.isnull().sum()`	Cuántos datos faltan por columna
Valores únicos	`df['col'].unique()`	Qué valores distintos hay en esa columna
Ancho a largo	`pd.melt()`	Convierte columnas-año en filas
Texto a número	`pd.to_numeric(s, errors='coerce')`	Convierte texto a número; si no puede, pone NaN
Unir tablas	`df1.merge(df2, on=['geo','anio'], how='outer')`	Junta dos tablas por columnas comunes
Agrupar	`df.groupby('col').mean()`	Agrupa filas y calcula la media (o sum, count...)
Filtrar filas	`df[df['col'] > 5]`	Quédate solo con las filas donde col sea mayor que 5
Quitar duplicados	`df.drop_duplicates()`	Elimina filas repetidas
Test Pearson	`pearsonr(x, y)`	Calcula r y p-value entre dos columnas
Guardar CSV	`df.to_csv('f.csv', index=False)`	Guarda la tabla como archivo CSV
Verificar duplicados	`df.groupby(['geo','anio']).size().value_counts()`	Comprueba si hay más de 1 fila por país/año

¿Cuánto vale tu trabajo?

Depende de dónde vivas.